

АННОТАЦИЯ

Отчет 64 с., 3 ч., 12 рис., 0 табл., 27 источн., 2 прил.

ИНСТРУМЕНТЫ РАЗРАБОТКИ, АВТОМАТИЗАЦИЯ, УПРАВЛЕНИЕ, ИНФРАСТРУКТУРА

Объект исследования — процессы и инструменты разработки, развертывания и эксплуатации прикладных сервисов в научных лабораториях с ограниченными вычислительными ресурсами и малочисленным штатом. Под прикладным сервисом в рамках данной работы понимается независимый программный или аппаратно-программный модуль, предназначенный для решения конкретной физико-технической задачи: от управления лабораторным оборудованием и сбора показаний до сложной математической обработки данных эксперимента.

Цель работы заключается в формировании методики разработки, развертывания и эксплуатации прикладных сервисов и создании программной платформы в качестве инкапсулированного инструментария, обеспечивающего её реализацию, с практической проверкой на базе инфраструктуры лаборатории №11 Института ядерной физики им. Г.И. Будкера СО РАН.

В ходе работы были проанализированы существующие процессы эксплуатации гетерогенного программного обеспечения, разработана платформа на основе принципов предметно-ориентированного моделирования, декларативного описания целевого состояния прикладного программного комплекса и методологии GitOps [1]. Практическая ценность подтверждена внедрением платформы в рабочие процессы лаборатории №11 ИЯФ СО РАН и успешной реализацией нескольких прикладных сервисов на её основе.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1. ОБЗОР И ПОСТАНОВКА ЗАДАЧИ	8
1.1. ОБЗОР ПРОГРАММНОЙ ИНФРАСТРУКТУРЫ	8
1.2. АНАЛИЗ СУЩЕСТВУЮЩЕЙ МОДЕЛИ ЭКСПЛУАТАЦИИ ПРИКЛАДНЫХ СЕРВИСОВ	12
1.3. ПОСТАНОВКА ЗАДАЧИ	20
2. КОНЦЕПТУАЛЬНОЕ ПРОЕКТИРОВАНИЕ	21
2.1. КОНЦЕПЦИЯ	21
2.2. СРЕДА ВЫПОЛНЕНИЯ	27
2.3. ПОРТАЛ ОПЕРАТОРА	30
2.4. СЦЕНАРИИ ВЗАИМОДЕЙСТВИЯ	35
3. РЕАЛИЗАЦИЯ	36
3.1. ВВЕДЕНИЕ К ПРИКЛАДНОМУ СЦЕНАРИЮ	36
3.2. ДЕКЛАРАТИВНОЕ ОПИСАНИЕ КОМПЛЕКСА	38
3.3. ОБЕСПЕЧЕНИЕ ВОСПРОИЗВОДИМОСТИ	40
3.4. ТРАНСПОРТНАЯ ИНФРАСТРУКТУРА	43
3.5. КОНВЕЙЕР АВТОМАТИЗАЦИИ И КОДОГЕНЕРАЦИИ	44
3.6. МЕНЕДЖЕР СЕРВИСОВ И СРЕДА ВЫПОЛНЕНИЯ	46
3.7. ПОРТАЛ ОПЕРАТОРА	55
3.8. ОГРАНИЧЕНИЯ РЕАЛИЗАЦИИ	58
ЗАКЛЮЧЕНИЕ	60
СПИСОК ЛИТЕРАТУРЫ	62
ПРИЛОЖЕНИЕ А. ПОТОКОВЫЙ СЕРВИС	65
ПРИЛОЖЕНИЕ Б. ПАКЕТНЫЙ СЕРВИС	68

ВВЕДЕНИЕ

Современные установки для проведения физических экспериментов, такие как коллайдер ВЭПП-2000 Института ядерной физики им. Г.И. Будкера СО РАН, представляют собой сложные научно-технические комплексы, предназначенные для проведения фундаментальных физических исследований.

Управление такими установками и анализ получаемой информации осуществляются посредством специализированных программных и аппаратных решений. В контексте данной работы под прикладным сервисом понимается независимый программный или аппаратно-программный модуль, предназначенный для решения конкретной физико-технической задачи: от управления лабораторным оборудованием и сбора показаний до сложной математической обработки данных эксперимента.

Программная инфраструктура многих научных лабораторий, включая лабораторию №11 ИЯФ СО РАН, часто формируется эволюционным путем: прикладные программные модули создаются и внедряются различными разработчиками для решения локальных прикладных задач. Подобный процесс развития, эффективный на ранних этапах, со временем приводит к возникновению существенного дисбаланса между возрастающей сложностью распределенных подсистем и применяемыми ручными методами их администрирования. В результате инфраструктура теряет способность к масштабированию, а её эксплуатация становится источником операционных рисков: сбоев в работе, некорректных результатов и повреждения экспериментального оборудования.

Объектом исследования в данной работе являются процессы и инструменты разработки, развертывания и эксплуатации прикладных сервисов в научных лабораториях с ограниченными вычислительными ресурсами и малочисленным штатом.

Предметом исследования являются методы построения и интеграции программной платформы путем внедрения практик GitOps в процессы разработки, развертывания и эксплуатации прикладных сервисов, а также подходы к декларативному описанию целевого состояния прикладного программного комплекса

и предметно-ориентированному моделированию для решения проблем гетерогенности и воспроизводимости в условиях ограниченности вычислительных ресурсов и специфики малочисленного штата специалистов.

Цель работы заключается в формировании методики разработки, развертывания и эксплуатации прикладных сервисов и создании программной платформы в качестве инкапсулированного инструментария, обеспечивающего её реализацию, с практической проверкой на базе инфраструктуры лаборатории №11 Института ядерной физики им. Г.И. Будкера СО РАН.

Для достижения поставленной цели были определены следующие задачи:

- Проанализировать существующие процессы эксплуатации прикладного программного обеспечения в лаборатории и сформулировать требования к программной платформе.
- Разработать концепцию и спроектировать архитектуру программной платформы.
- Разработать программные компоненты платформы.
- Провести практическую проверку разработанной платформы в инфраструктуре лаборатории.

Практическая значимость работы заключается в модернизации процессов разработки, внедрения, сопровождения и эксплуатации прикладных сервисов лаборатории №11 ИЯФ СО РАН. Учет специфики лаборатории позволяет интегрировать систему без нарушения существующих рабочих процессов. Сформулированные выводы и разработанные решения, оформленные в виде программной платформы, могут быть использованы в других лабораториях со схожими потребностями.

Работа имеет следующую структуру. В первой главе приводится обзор программной инфраструктуры лаборатории, анализируются существующие проблемы и формулируются требования к решению. Во второй главе разрабатываются концепция и архитектура программной платформы. Третья глава посвящена реализации ключевых компонентов платформы. В заключение подводятся итоги работы и намечаются пути дальнейшего развития. В приложениях представлены

примеры практической реализации прикладных сервисов потоковой (Приложение А) и пакетной (Приложение Б) обработки данных на базе разработанной платформы.

1. ОБЗОР И ПОСТАНОВКА ЗАДАЧИ

Данная глава посвящена анализу текущего состояния процессов разработки, внедрения и эксплуатации программного обеспечения в научных лабораториях. Основная цель обзора — системно описать существующую реальность эксплуатации ИТ-инфраструктуры в условиях ограниченных ресурсов.

В качестве представителя данного класса исследовательских центров рассматривается лаборатория №11 ИЯФ СО РАН, обслуживающая коллайдер ВЭПП-2000 (см. [2]). Выявление проблем, анализ существующих подходов и формирование финальных требований к решению базируются на практическом опыте работы с этой конкретной лабораторией. При этом сделанные выводы характерны и для других научных коллективов, сталкивающихся с аналогичными проблемами.

1.1. ОБЗОР ПРОГРАММНОЙ ИНФРАСТРУКТУРЫ

1.1.1. РОЛЬ ПРОГРАММНЫХ СЕРВИСОВ

В научных лабораториях прикладные программные сервисы выполняют разнообразные задачи, обслуживая проводимые эксперименты на всех стадиях. В контексте данной работы под программным сервисом понимается любая единица исполнения, решающая одну прикладную задачу: от высокоуровневых аналитических модулей до программных драйверов, непосредственно взаимодействующих с экспериментальным оборудованием. К фундаментальным задачам сервисов относятся:

- Сбор и организация данных.
- Аналитическая обработка данных.
- Автоматизированное управление режимами работы физической установки.
- Визуальная интерпретация полученных результатов.

При системном подходе к автоматизации совокупность прикладных сервисов реализует цифровую модель исследуемого объекта или явления. Наличие такой модели способствует более качественному автоматизированному анализу данных. Сама исполняемая совокупность сервисов далее будет называться прикладным программным комплексом.

Программный комплекс функционирует в рамках специализированной операционной среды, которая позволяет его разрабатывать, обслуживать и эксплуатировать. Далее такая среда будет называться программной инфраструктурой. Отношение между цифровой моделью, комплексом и инфраструктурой продемонстрировано на рисунке 1.

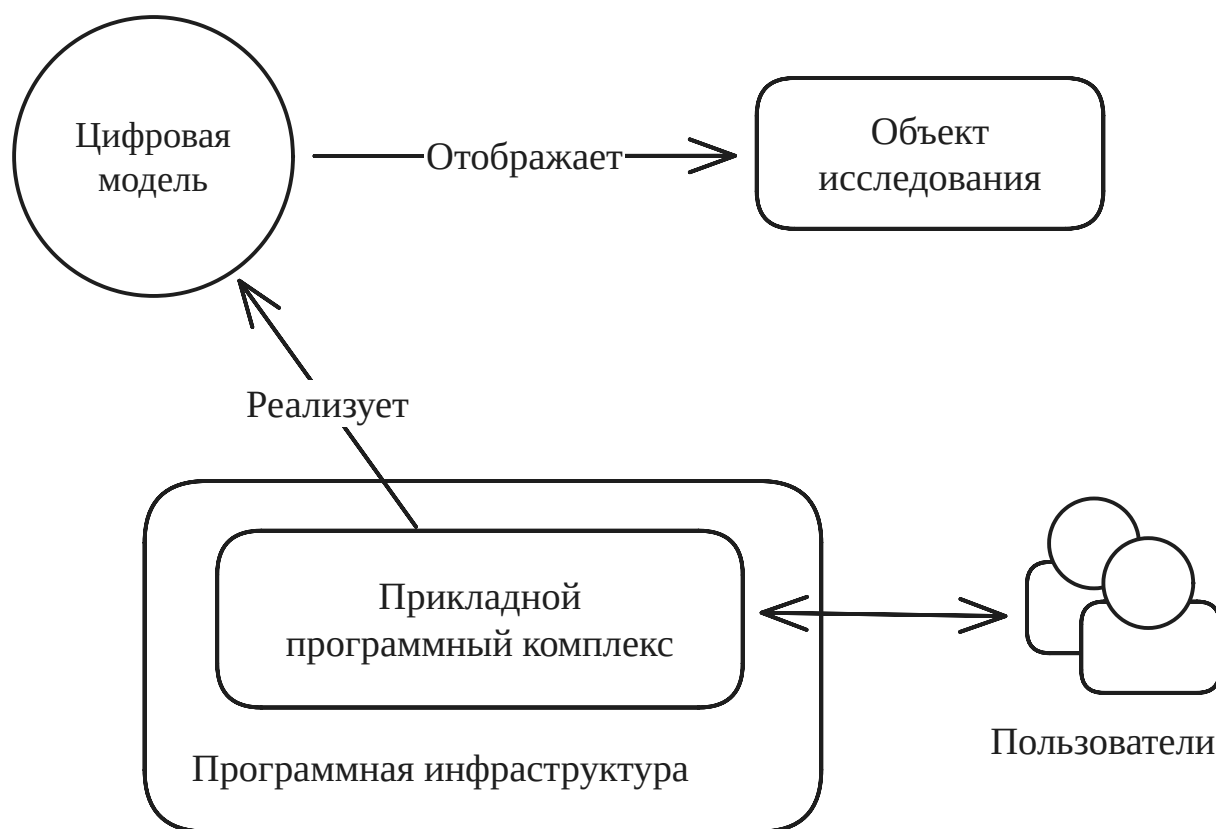


Рисунок 1 – Структурно-логическая схема программной инфраструктуры лабораторного комплекса

1.1.2. ЖИЗНЕННЫЙ ЦИКЛ ПРОГРАММНЫХ СЕРВИСОВ

Период разработки и эксплуатации прикладных сервисов состоит из нескольких этапов. Типовой жизненный цикл сервиса приведен на рисунке 2.

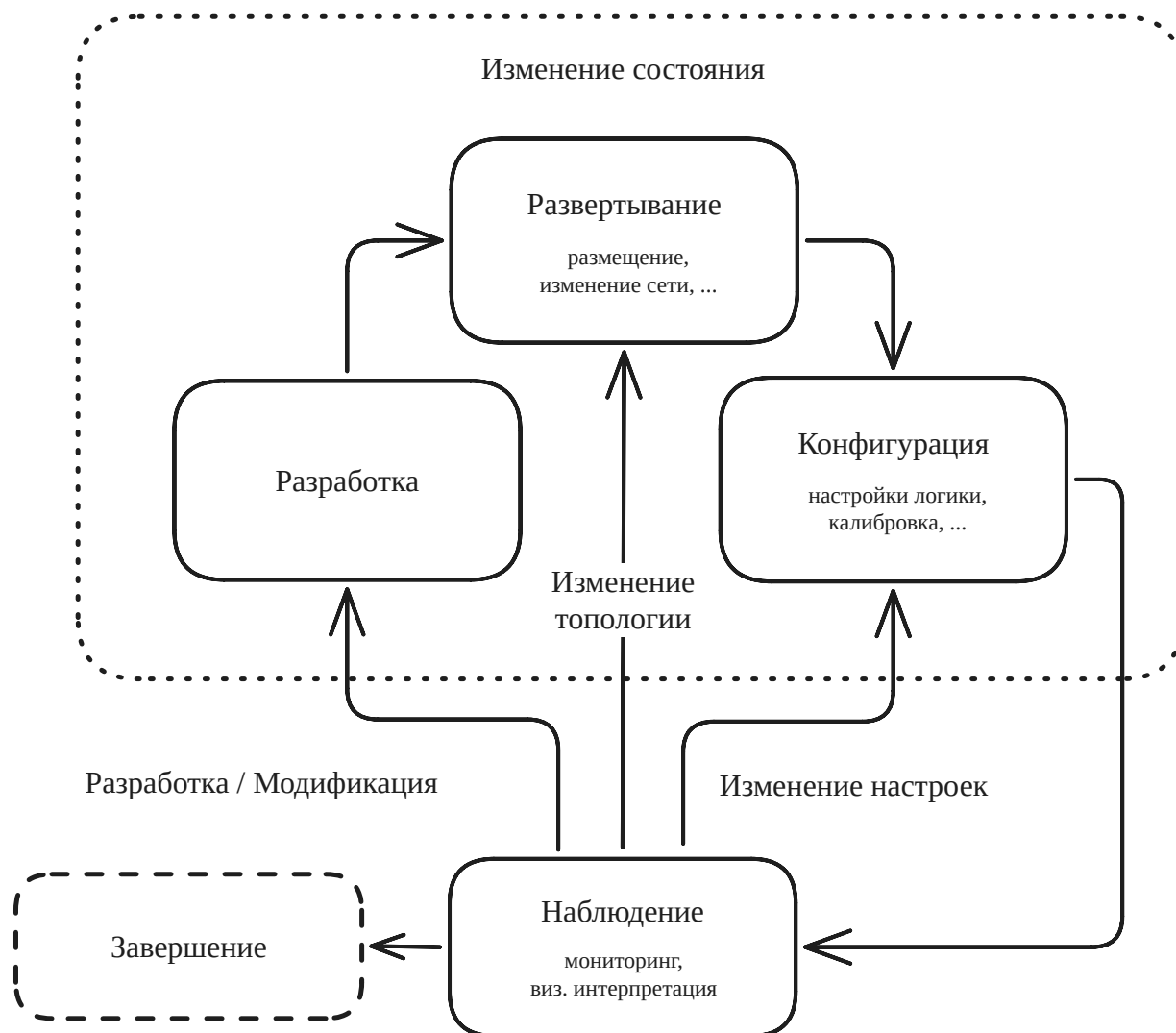


Рисунок 2 – Жизненный цикл программных сервисов

1.1.3. КАТЕГОРИИ ПОЛЬЗОВАТЕЛЕЙ И ИХ ФУНКЦИИ

При взаимодействии с программным комплексом пользователям, как правило, присваиваются следующие роли:

- Разработчик программного комплекса (отвечает за создание прикладной логики, разработку НМИ (человеко-машинного интерфейса) и настройку взаимодействия с оборудованием).
- Администратор программного комплекса (отвечает за развертывание, мониторинг и поддержку бесперебойной работы сервисов).
- Оператор физической установки (определяет ход проведения эксперимента посредством визуальной интерпретации результатов работы программного комплекса и его конфигурирования).

Каждая роль определяет набор пользовательских сценариев:

- Разработчик:
 - Реализация прикладной логики сервисов (вычислительных алгоритмов, сбора и обработки данных) с использованием предпочтительных технологий.
 - Разработка человеко-машинного интерфейса для конфигурации и управления сервисом.
 - Разработка документации к сервисам.
 - Регистрация сервисов в системе.
 - Хранение и распространение программного кода.
- Администратор:
 - Инициализация и размещение конфигурационных файлов.
 - Ввод сервисов в эксплуатацию.
 - Разрешение технических конфликтов, возникающих в процессе эксплуатации программного комплекса (например, несоответствие актуального и целевого состояний).
- Оператор:
 - Изучение документации.
 - Конфигурирование программного комплекса.
 - Анализ результатов работы лабораторного комплекса.

1.1.4. ОСОБЕННОСТИ ЭКСПЛУАТАЦИИ ПРИКЛАДНЫХ СЕРВИСОВ В ЛАБОРАТОРНЫХ УСЛОВИЯХ

В условиях малочисленного штата пользователи могут одновременно выполнять функции двух или всех трех ролей. Кроме того, функционирование прикладного программного комплекса в лабораторных условиях характеризуется следующими особенностями:

- Локальная инфраструктура. Фиксированное количество вычислительных узлов, расположенных в непосредственной близости к объекту исследования.

- Гетерогенные сервисы. Как отмечалось ранее, прикладные сервисы разрабатываются различными специалистами с использованием разнообразных технологий.
- Длительный срок эксплуатации сервисов. Некоторые сервисы предполагают непрерывный режим работы в течение длительного времени (год и более). При этом параллельно с ними существуют сервисы, работающие периодически, но с большими объемами данных.
- Большой объем существующих разработок, требующих обеспечения обратной совместимости.

1.2. АНАЛИЗ СУЩЕСТВУЮЩЕЙ МОДЕЛИ ЭКСПЛУАТАЦИИ ПРИКЛАДНЫХ СЕРВИСОВ

Анализ проблем целесообразно проводить по классам, рассматривая и решая задачи по порядку:

- Управление исходным кодом и среда разработки.
- Автоматизация сборки, тестирования и развертывания.
- Управление состоянием программного комплекса.
- Координация времени выполнения.
- Межсервисное взаимодействие.
- Человеко-машинный интерфейс.

Уже на этом этапе становится очевидно, что решение на первый взгляд разнородных задач вызывает затруднения у научных команд небольших размеров. Рассмотрим каждый класс задач отдельно по следующим аспектам:

- Назначение и роль в инфраструктуре.
- Текущая практическая реализация.
- Выявленные недостатки и проблемы.
- Анализ существующих технологических подходов.

1.2.1. УПРАВЛЕНИЕ ИСХОДНЫМ КОДОМ И СРЕДА РАЗРАБОТКИ

В условиях небольшой команды и дефицита узкопрофильных специалистов грамотное управление исходным кодом и обеспечение совместной разработки критически важны. Это необходимо для эффективного распределения задач, надежного контроля версий и устранения дублирования функциональности при сосуществовании гетерогенных сервисов на различных языках и платформах. Системный подход позволяет минимизировать риски ошибок, ускорить разработку, повысить качество исходного кода и обеспечить долгосрочную поддерживаемость сервисов.

В рамках существующего подхода каждый сервис сопровождается независимо от остальных. Отсутствует стандартный подход к управлению внутренними зависимостями. Среда разработки не зафиксирована: разработчику необходимо найти и установить нужные программные компоненты и инструменты самостоятельно, попутно решая возникающие конфликты (например, несоответствие версий).

Такой подход влечет за собой ряд проблем:

- Большой объем дублирующейся функциональности.
- Высокий порог входа для разработчиков.
- Отсутствие воспроизводимости среды разработки.

Современные индустриальные практики предполагают стандартизацию среды разработки посредством ее контейнеризации (см. [3], [4]). Помимо решения проблемы воспроизводимости и снижения порога входа для новых специалистов, такой подход открывает новые возможности для создания удаленных окружений разработки (RDE, см. [5], [6]).

Для управления зависимостями в гетерогенной среде выделяются два основных подхода:

- Поддержка инструментов управления артефактами (программными библиотеками) и их распространения.

- Совместное размещение исходного кода (см. [7], [8]) с целью совместной компиляции и версионирования.

1.2.2. АВТОМАТИЗАЦИЯ СБОРКИ, ТЕСТИРОВАНИЯ И РАЗВЕРТЫВАНИЯ

Автоматизация сборки, тестирования и развертывания критична для обеспечения надежности и скорости ввода сервисов в эксплуатацию (см. [9]). В условиях гетерогенности сервисов, автоматизация снижает вероятность ошибок, обеспечивает воспроизводимость процессов и позволяет минимизировать ручной труд небольшой команды. Грамотно выстроенный процесс интеграции значительно ускоряет процесс проверки новых концепций, что критично в экспериментальных лабораториях.

В рамках существующего подхода все инструменты, включая систему сборки, изолируются одним проектом, т. е. создаются и сопровождаются независимо друг от друга. Стандартной системы управления внешними зависимостями нет. Большинство проектов полагаются на наличие системных программных пакетов. Развертывание сервисов осуществляется администратором в ручном режиме.

Сопутствующие такому подходу проблемы включают:

- Невозможность разработки и использования общих внутренних инструментов.
- Невоспроизводимость и риск конфликтов с рабочей средой (например, для сборки проекта требуется одна версия библиотеки, а в системе установлена другая).
- Высокая нагрузка на администратора и длительный процесс ввода сервисов в эксплуатацию.

К индустриальным подходам к решению данных и других проблем относятся:

- Совместное размещение исходного кода и внедрение различных инструментов для организации хранения и сборки большого количества программных модулей.

- Использование комплексных инструментов управления внешними зависимостями.
- Использование распределенных оркестраторов (см. [10]) и систем управления инфраструктурой (см. [11]).

1.2.3. УПРАВЛЕНИЕ СОСТОЯНИЕМ ПРОГРАММНОГО КОМПЛЕКСА

Состояние прикладного программного комплекса формируется из множества состояний входящих в него подсистем, включающего и прикладные сервисы. Состояние подсистемы определяется ее исходным кодом, конфигурацией, параметрами развертывания и другой информацией, специфической для каждой конкретной подсистемы.

В процессе функционирования программного комплекса выделяется два состояния, сосуществующих в один момент времени:

- **Административное состояние системы** — желаемое состояние системы в определенный момент времени, непрерывно формируемое оператором в процессе управления.
- **Оперативное состояние системы** — фактическое, наблюдаемое в реальном времени состояние работающего программно-аппаратного комплекса.

Фундаментальной задачей разработчиков и администраторов является приведение оперативного состояния в соответствие административному.

В существующей практике управления комплексом доминирует императивный подход, основанный на концепции интерактивного ручного администрирования. Оператор осуществляет управление системой локально, транслируя управляющие воздействия в виде команд через разрозненные графические интерфейсы прикладных сервисов или путем прямого редактирования конфигурационных файлов в файловой системе конкретных вычислительных узлов. Параметры применяются службами непосредственно в оперативную память или считываются в момент перезапуска. При этом промежуточные состояния комплекса централизованно не фиксируются, из-за чего история изменений мгновенно утрачивается.

Подобный подход включает следующие очевидные недостатки:

- Высокая нагрузка на администратора по управлению оперативным состоянием прикладного программного комплекса. Для разрешения возникающих сбоев и перезагрузок требуется вручную перенастраивать все поврежденные сервисы.
- Отсутствие версионирования. Крайне сложно осуществить переход к состоянию, при котором эксперимент показал наилучшие результаты.

В индустриальной практике для решения подобных задач традиционно применяются развитые коммерческие SCADA-системы и специализированные распределенные хранилища конфигураций, интегрированные с тяжелыми системами оркестрации вычислительных ресурсов. Эти подходы опираются на использование централизованных баз данных, проприетарных протоколов связи и развитых HMI систем для непрерывного контроля и синхронизации параметров распределенных комплексов.

1.2.4. КООРДИНАЦИЯ ВРЕМЕНИ ВЫПОЛНЕНИЯ

Управление состоянием сервисов включает в себя их распределение по вычислительным узлам, запуск, остановку, доставку актуальной конфигурации и автоматический контроль работоспособности. Кроме того, в процессе эксплуатации комплекса требуется оперативная и достоверная информация о текущем состоянии каждого компонента.

В существующей практике координация компонентов осуществляется полностью в ручном режиме в рамках небольшого локально-распределенного кластера лаборатории. Разработчики и администраторы самостоятельно распределяют прикладные сервисы по вычислительным узлам и вручную запускают и останавливают процессы. Оперативный контроль работоспособности комплекса также выполняется напрямую: администратор вынужден удаленно подключаться к каждому конкретному узлу кластера и оценивать состояние системы по косвенным признакам, отслеживая идентификаторы запущенных процессов средствами операционной системы.

Подобный подход приводит к возникновению ряда критических проблем:

- Высокая нагрузка на администратора и концентрация критических знаний у ограниченного числа лиц из-за гетерогенности среды выполнения.
- Отсутствие механизмов автоматического восстановления работоспособности критических сервисов.
- Отсутствие воспроизводимости рабочей среды (нельзя быстро развернуть программный комплекс на свежем оборудовании или в тестовой среде).
- Сложность масштабирования.

Для устранения этих проблем в индустрии были разработаны различные архитектурные подходы. В целом их можно разделить на два класса:

- Подсистемы автоматизации и конфигурационного управления инфраструктурой, ориентированные на декларативное описание вычислительных ресурсов и автоматическую доставку артефактов на целевые узлы.
- Оркестраторы сред выполнения, обеспечивающие автоматическое распределение процессов по кластеру, контроль их жизненного цикла и динамическое восстановление при сбоях.

Каждый из этих подходов решает свой изолированный круг задач, и при построении крупных распределенных систем разработчики, как правило, сочетают элементы обоих классов.

1.2.5. МЕЖСЕРВИСНОЕ ВЗАИМОДЕЙСТВИЕ

Система межсервисного взаимодействия определяет возможности управления, обработки данных и применения различных вычислительных моделей, упрощающих разработку сложных алгоритмов (см. [12], [13]). Выбор подходов и протоколов обмена данными напрямую определяет отзывчивость, пропускную способность и общую производительность гетерогенного комплекса.

Помимо интерактивного управления типа «точка-точка», основанного на использовании проприетарных протоколов, преобладающей моделью взаимодействия в лабораторных комплексах является событийно-ориентированная

архитектура. Основной информационный поток генерируется устройствами, непосредственно взаимодействующими с объектом исследования; он включает данные о состоянии подсистем, направляемые в один или несколько информационных каналов.

В существующей модели каналы данных обслуживаются собственным брокером сообщений с внутренним текстовым протоколом. При этом структура каналов и тип передаваемых сообщений не стандартизированы.

Возникающие проблемы включают:

- Избыточность текстовых протоколов передачи данных.
- Высокая связность сервисов, провоцирующая риски каскадных сбоев во всей системе при отказе одного узла.
- Отсутствие возможности эволюции структуры полезной нагрузки.
- Деградация производительности с внедрением новых аналитических сервисов.

В развитой индустриальной практике (см. [14]) взаимодействие между сервисами разделяется на два уровня. На первом уровне происходит обмен информацией с устройствами, которые взаимодействуют непосредственно с изучаемым физическим комплексом. На втором уровне происходит обмен информацией между аналитическими сервисами, выполняющими основной объем вычислений.

1.2.6. ЧЕЛОВЕКО-МАШИННЫЙ ИНТЕРФЕЙС

В лабораторной эксплуатации графический интерфейс — это основной метод управления и наблюдения. Качество предоставляемых интерфейсов напрямую влияет на работу оператора, скорость принятия решений и результаты экспериментальной деятельности.

В существующей модели графический интерфейс интегрируется непосредственно в сервис, который поставляется в форме локального настольного приложения. Хотя унифицированные требования к содержанию интерфейса отсутствуют, большинство сервисов включают инструменты для конфигуриро-

вания, запуска, остановки, мониторинга оперативного состояния и визуализации результатов работы.

Проблемы и недостатки, сопряженные с таким подходом:

- Ограничение выбора технологий разработки: реализация интерфейса на традиционных языках (например, C++ или Java) является отработанным решением, в то время как интеграция графических элементов с современными языками (например, Go или Lua) существенно затруднена.
- Отсутствие многопользовательского режима и удаленного доступа к сервису.
- Ограничение среды выполнения сервиса вычислительными ресурсами рабочей станции оператора.
- Дополнительные временные затраты на ручную реализацию типовых элементов управления.

В индустриальной практике с целью минимизации времени разработки, повышения производительности и обеспечения многопользовательского доступа вычислительную часть сервиса отделяют от графического интерфейса. Для управления сервисами применяются современные проприетарные инструменты построения НМІ, включающие элементы конфигурирования и визуализации.

1.2.7. ВЫВОД

Без должного контроля и обслуживания совокупность прикладных сервисов превращается в фрагментированную и слабоуправляемую среду, что снижает скорость разработки и подвергает работу лабораторного комплекса критическим рискам.

Для большинства рассматриваемых классов задач существует надежное, проверенное временем, универсальное решение. Более того, новые подходы появляются практически каждый день. Однако разобраться во всех существующих системах невозможно. Особенно остро проблема возникает в небольших научных лабораториях, где, в условиях отсутствия экспертизы, требуемой для сопровож-

дения индустриальных систем, попытки использования подобных решений могут приводить к существенному росту операционных издержек.

1.3. ПОСТАНОВКА ЗАДАЧИ

На основе проведенного анализа существующей программной инфраструктуры лаборатории, ключевой практической задачей настоящей работы является разработка и практическая проверка программной платформы для методичного управления жизненным циклом гетерогенных прикладных сервисов.

Разрабатываемое решение должно функционировать в условиях ограниченных вычислительных и кадровых ресурсов, целенаправленно минимизируя операционную нагрузку. Для достижения этой цели платформа должна инкапсулировать системные механизмы, скрывая инфраструктурную сложность от пользователей за предметно-ориентированными интерфейсами.

2. КОНЦЕПТУАЛЬНОЕ ПРОЕКТИРОВАНИЕ

В данной главе разрабатывается концептуальная модель программной платформы, представляющая собой комплексное решение проблем, изложенных в предыдущем разделе, для лабораторий с ограниченными ресурсами.

2.1. КОНЦЕПЦИЯ

Архитектура программной платформы представляет собой совокупность функционально независимых подсистем, которые объединяются в единый инструментальный комплекс и унифицируются за единым предметно-ориентированным интерфейсом. С точки зрения пользователя, платформа выступает в роли абстрактного «черного ящика», инкапсулирующего внутреннюю операционную сложность и включающего всё необходимое для автоматизации процессов разработки, внедрения и эксплуатации прикладного программного комплекса. В связи с этим, дальнейшее описание структуры и механизмов работы платформы будет изложено помодульно, последовательно раскрывая каждую из интегрированных подсистем.

2.1.1. ПРЕДМЕТНАЯ МОДЕЛЬ

Для взаимодействия с платформой применяется специальный декларативный язык. Средствами этого языка формируется предметная модель, которая полностью определяет поведение платформы.

Предметная модель программного комплекса формализует структуру, ограничения и функциональные возможности цифровой модели (см. раздел 1.1). Она состоит из формальных объектов предметной области и связей между ними, описанных с помощью знакомых инструментов алгебры множеств и логики. Например, в лаборатории №11 предметная модель состоит из формальных объектов, описывающих устройство коллайдера ВЭПП-2000.

В качестве примера приведена структура формального объекта «Материал», с помощью которого в лаборатории представляется часть свойств используемых физических объектов:

```
// С помощью пакетов, объекты формируются в логические группы.  
// Объекты из одного пакета могут быть использованы в других  
// пакетах.  
package mat  
  
// Символом "#" обозначаются формальные объекты.  
#Material: {  
    // Плотность любого материала - строго положительное  
    // вещественное число.  
    density: >0.0  
    relativePermeability: >0.0  
}
```

Подобным образом могут быть описаны различные предметные области. Каждая лаборатория может представить один и тот же физический объект в удобном для нее виде.

Для настройки платформы, предметная модель расширяется дополнительными абстракциями. Они выступают упрощенным интерфейсом к сложным подсистемам, которые платформа скрывает внутри себя. Эти абстракции будут рассмотрены далее, при введении соответствующих подсистем.

2.1.2. СОСТОЯНИЕ

С помощью предметной модели пользователь платформы описывает множество **допустимых состояний** прикладного программного комплекса. Так, в примере выше, с помощью предметной модели множеством возможных значений для плотности материала были выбраны положительные действительные числа.

Для обобщения понятия состояния сервиса, платформа предлагает пользователю следующую абстракцию:

```
package dom  
  
...  
  
#Config: {
```

```

// Поле имеет логический тип и принимает ложное
// значение по умолчанию.
active:      bool | *false
deployment: #Deployment

// "... " означает, что объект может быть дополнен
// другими свойствами и ограничениями.
...
}

```

Формальный объект `Config` включает два параметра, которые будут входить в состояние любого сервиса. Параметр `active` определяет желаемый статус сервиса, т.е. включен сервис или выключен. Параметр `deployment` определяет конфигурацию развертывания сервиса и будет рассмотрен более детально в соответствующем разделе.

Прикладные сервисы наследуют базовый объект `dom.#Config`, расширяя его специфичными для физического оборудования параметрами. Так, модель конфигурации для расчета магнитного поля многокатушечного соленоида принимает следующий вид:

```

package mf

// Далее будут использоваться ранее
// введенные объекты предметной области`.
import "github.com/blabtm/v2k/dom"
import "github.com/blabtm/v2k/dom/mat"

#Yoke: {
    material: mat.#Material
}

#Coil: {
    material:      mat.#Material
    currentAmp:    >0.0
    numberOfTurns: >0
}

// "&" - это оператор унификации. В данном случае,
// свойства и ограничения "dom.#Config" объединяются с новыми
// свойствами и ограничениями.
#Config: dom.#Config & {
    yoke: #Yoke
}

```

```
coils: {  
  nbti: #Coil  
  nbsn: #Coil  
  comp: #Coil  
}  
}
```

Имея такое описание, пользователи и платформа знают не только необходимые для работы сервиса параметры, но и множество допустимых значений.

Административное состояние системы — элемент ранее заданного множества допустимых состояний, который соответствует желаемому состоянию системы в определенный момент времени.

Оперативное состояние системы — это фактическое, наблюдаемое в реальном времени состояние работающего программно-аппаратного комплекса.

Возвращаясь к примеру, административное состояние системы — конкретные значения параметров, определенных формальным объектом `Config`. При этом оперативное состояние системы может отличаться, если, например, эти конкретные значения еще не были доставлены до сервиса или он не успел их применить.

Задача платформы заключается в приведении оперативного состояния в соответствие с административным, гарантируя при этом, что административное состояние всегда удовлетворяет ограничениям предметной модели (принадлежит множеству допустимых состояний). Визуально этот процесс изображен на рисунке 3.



Рисунок 3 – Процесс перехода между состояниями

2.1.3. СИНХРОНИЗАЦИЯ РЕСУРСОВ

Под ресурсами понимаются любые сущности, представленные в виде файлов (текстовых или бинарных), которые описывают желаемый вид, настройки и поведение компонентов экспериментальной установки. Другими словами, совокупность ресурсов описывает административное состояние комплекса.

Платформа спроектирована так, что все аспекты функционирования прикладного программного комплекса (его состояния) могут быть представлены в виде ресурсов, которые, в свою очередь, организуются, хранятся и версионизируются совместно. Таким образом, для синхронизации состояния программного комплекса требуется синхронизировать все описывающие его ресурсы.

Для работы комплекса требуются разнообразные формы представления состояния:

- Исходный код прикладных сервисов.
- Файлы конфигурации оборудования и программного обеспечения.

- Описания вычислительных сред и инфраструктуры.
- Ресурсы внешних подсистем (например, дашборды и настройки оповещений Grafana).

Кроме приведенных выше, существует еще множество форм представления, используемых внутренними подсистемами оркестрации, планирования, визуализации и т.д. Платформа берет на себя управление всеми этими ресурсами и скрывает сложные правила их сборки, верификации и развертывания за своим интерфейсом (см. [15]). Образно этот процесс изображен на рисунке 4.

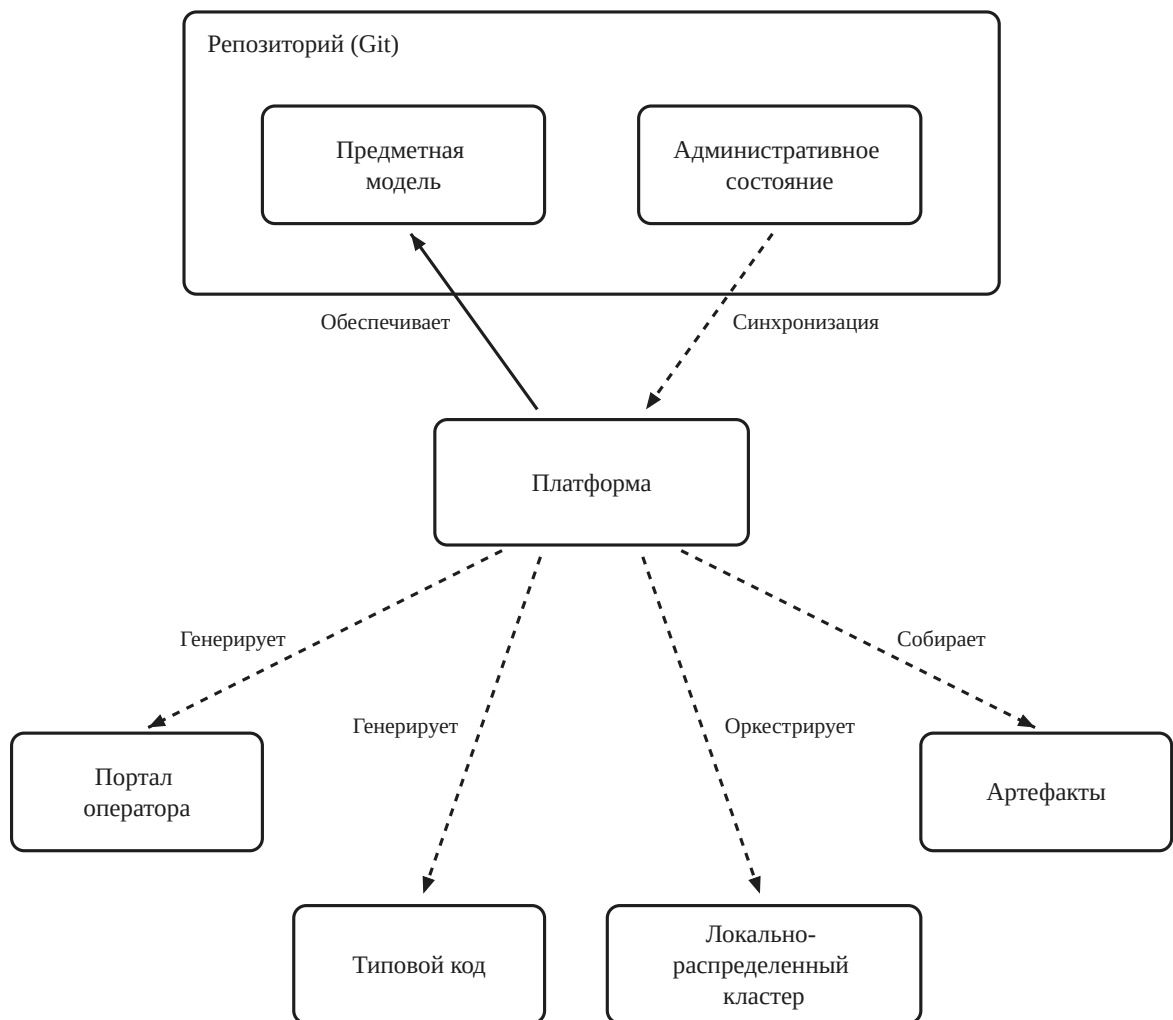


Рисунок 4 – Синхронизация состояния

2.2. СРЕДА ВЫПОЛНЕНИЯ

Имея примерное представление о том, что должна делать платформа с точки зрения пользователя, можно более подробно рассмотреть интерфейсы, которые она предоставляет для доступа к своим подсистемам.

Как было обозначено ранее, одним из функциональных требований к платформе является отображение фиксированного множества сервисов на фиксированное множество локальных вычислительных узлов. Для этого в предметную модель включается декларативное описание узлов локально-распределенного кластера, согласно парадигме IaC [16] («инфраструктура как код»):

```
#Node: {
  name!: =~"^[a-z]+[a-z0-9]*$" // уникальное имя узла
  host!: string & net.IPv4      // адрес узла в локальной сети
  port: >0 & <=65535 | *22     // SSH порт для подключения к узлу
}

#Cluster: [#Node]

cluster: #Cluster & [
  {
    name: "master"
    host: "192.168.0.37"
    port: 22
  },
]
```

С помощью формального объекта `Node` пользователь предоставляет платформе всю информацию для взаимодействия с кластером. Сам кластер, описываемый формальным объектом `Cluster` представляет собой множество именованных узлов.

С помощью объекта `cluster` администратор перечисляет имеющиеся в распоряжении лаборатории вычислительные узлы. Этот объект является частью состояния системы. Платформа может использовать эту информацию в самых разнообразных целях:

- Отображать сервисы на вычислительные узлы.
- Разворачивать и оркестрировать саму себя.

- Отслеживать работоспособность вычислительных узлов.

Внутри платформы эти задачи могут делегироваться различным промышленным системам, каждая из которых требует это же описание множества узлов в своем формате.

Для отображения сервиса на узел, в его состояние включается параметр `deployment`, который уже встречался в примере ранее. Тип этого параметра описывается формальным объектом `Deployment`:

```
#Deployment: {
  // Данная конструкция формирует список имен узлов
  // описанного ранее кластера и проверяет, входит ли
  // указанное имя в этот список.
  //
  // Параметр не является обязательным, что
  // отражается символом "?".
  node?: string & or([for n in cluster {n.name}])
}
```

Параметр `node` содержит имя узла и демонстрирует важную функциональную возможность предметной модели. С помощью приведенной выше конструкции, которая состоит из двух операторов алгебры множеств (объединение и пересечение), накладывается простое ограничение на принадлежность узла кластеру.

Ранее было сказано, что лабораторный комплекс эксплуатирует гетерогенные прикладные сервисы. Это означает, помимо прочего, что сервисы разрабатываются для функционирования в определенной среде. Наиболее популярные в научных лабораториях среды выполнения:

- Среда операционной системы.
- Контейнеризированная среда.
- Микроконтроллерная среда (отсутствие среды как таковой).

Для того, чтобы платформа могла управлять гетерогенными сервисами, пользователю предоставляется дополнительная абстракция сервиса, которая не входит в его состояние (т.к. тип сервиса не меняется в процессе эксплуатации):

```
#Driver: "docker" | "system" | "firmware"

#Service: {
    // Этот обязательный параметр (символ "!") может
    // принимать одно из значений выше.
    driver!: #Driver
}
```

Продолжая пример с сервисом расчета магнитного поля, дескриптор будет выглядеть следующим образом:

```
package mf

import "github.com/blabtm/v2k/dom"

dom.#Service & {
    driver: "docker"
}
```

Дескриптор определяется разработчиком сервиса и содержит конкретное значение параметра `driver`, который определяет целевую среду выполнения. В данном случае, сервис разработан для функционирования в контейнеризированной среде.

Обладая информацией о типе среды выполнения и целевом вычислительном узле платформа автоматически определяет состав необходимых артефактов, а также способ и место их развертывания. На основе этой же информации платформа осуществляет доставку нового состояния (конфигурации) до запущенного сервиса. Образно процесс синхронизации состояния сервисов изображен на рисунке 5.

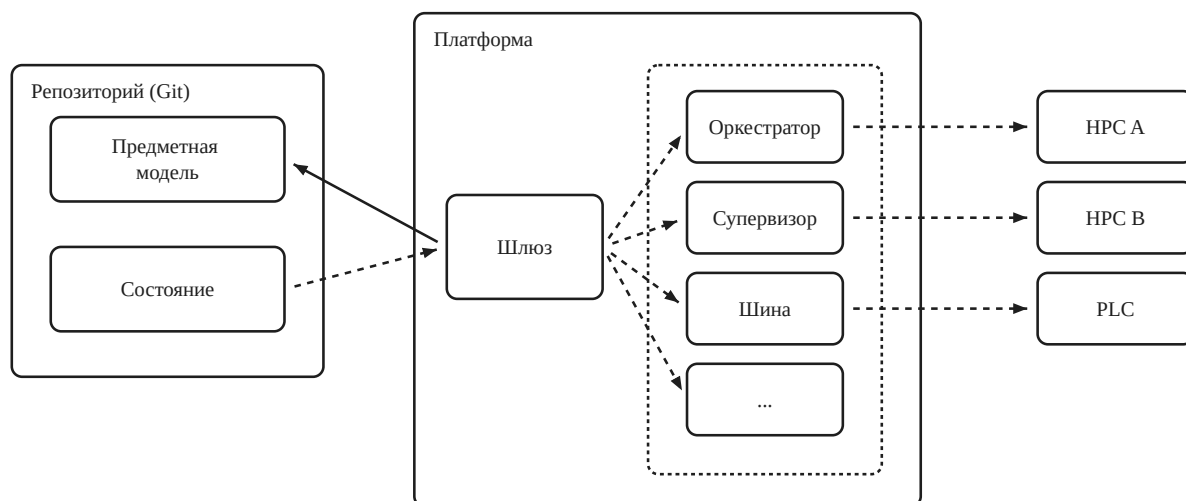


Рисунок 5 – Синхронизация состояния сервисов

Различные механизмы выполнения (оркестраторы контейнеров, системные супервизоры, скоростные шины данных и т.д., изображенные на схеме выше) полностью скрыты от пользователя за единым декларативным интерфейсом платформы и при необходимости могут быть заменены без модификации предметной модели или прикладного кода.

2.3. ПОРТАЛ ОПЕРАТОРА

Основное взаимодействие с прикладным программным комплексом осуществляют операторы в процессе выполнения повседневных задач наблюдения и управления. Для них среда выполнения и предметная модель не имеют большого значения. Оператору важно, чтобы система была наблюдаема и управляема.

2.3.1. НАБЛЮДАЕМОСТЬ

Под наблюдаемостью понимается фундаментальное свойство системы, характеризующее возможность всесторонней оценки её внутреннего состояния, динамики функционирования и происходящих событий на основе анализа доступных внешних (выходных) данных. Далее будет рассматриваться одна из ключевых компонент наблюдаемости: подсистема мониторинга.

2.3.1.1. МОНИТОРИНГ

Для обеспечения оперативного наблюдения за ходом физического эксперимента и состоянием аппаратных комплексов требуется гибкая среда визуализации временных рядов и телеметрических потоков. В состав платформы в качестве подсистемы первого класса интегрирована аналитическая система Grafana (см. [17], [18]), являющаяся общепринятым индустриальным стандартом.

Поскольку Grafana предоставляет пользователям интуитивно понятный и развитый графический интерфейс для интерактивного анализа данных, платформа не изолирует эту подсистему за собственными представлениями. Вместо этого пользователю доступен весь её штатный функционал. Что платформа полностью скрывает внутри себя, так это операционную сложность управления ресурсами Grafana, которые в рамках предлагаемой архитектуры рассматриваются как неотъемлемая часть общего состояния прикладного программного комплекса. К базовым управляемым ресурсам подсистемы мониторинга относятся:

- Источники данных — конфигурации сетевого подключения к базам данных и брокерам сообщений. Они определяются один раз на уровне всей платформы.
- Информационные панели — графические экраны, содержащие метрики и графики. Они располагаются непосредственно в каталогах тех прикладных сервисов, к которым логически относятся, что реализует принцип локальности владения.
- Логические папки — структуры для группировки панелей и разграничения прав доступа к ним.

Традиционный подход к администрированию подобных систем предполагает ручное создание графических панелей через веб-интерфейс, что ведет к возникновению операционных рисков, потере воспроизводимости и отсутствию версионирования. Разрабатываемая платформа переводит управление мониторингом в другую парадигму. Все ресурсы Grafana декларативно описываются в

виде текстовых файлов, версионизируются в едином репозитории, рядом с кодом сервисов, и синхронизируются платформой автоматически.

В процессе синхронизации платформа берет на себя решение следующих инфраструктурных задач, скрывая их от пользователя:

- Интерполяция сетевых параметров. Платформа автоматически подставляет актуальные IP-адреса, порты и учетные данные целевых вычислительных узлов в декларативные манифесты источников данных, изолируя разработчика от специфики конкретного контура развертывания.
- Динамическая инъекция вычислительных схем. Ключевой проблемой при организации потокового мониторинга является необходимость декодирования строго типизированных бинарных сообщений реального времени на стороне клиента визуализации. Платформа автоматически перехватывает шаблоны информационных панелей и внедряет в них актуальные спецификации из репозитория. Благодаря такому подходу, при изменении структуры физических сигналов или переносе сервиса на другой вычислительный узел, оператору не требуется перенастраивать графические интерфейсы вручную.

Платформа гарантирует автоматическое приведение оперативного состояния панелей мониторинга к административному состоянию, зафиксированному в репозитории (см. [19]).

2.3.2. УПРАВЛЕНИЕ

Если подсистема мониторинга отвечает за пассивное наблюдение и оценку оперативного состояния, то подсистема управления предоставляет оператору инструменты для активного изменения административного состояния комплекса.

В традиционных индустриальных SCADA-системах построение человеко-машинного интерфейса (HMI) сопряжено со значительными трудностями. Во-первых, визуальное конструирование графических экранов и связывание их с тегами данных вручную – это сложный и ресурсоёмкий процесс, требующий отдельной разработки. При изменении структуры прикладных параметров (пред-

метной модели) все связанные визуальные формы приходится перенастраивать заново. Во-вторых, большинство классических НМІ-интерфейсов функционируют в виде локальных настольных приложений, что физически привязывает оператора к аппаратному пульта управления установкой.

Разрабатываемая платформа полностью скрывает процесс построения человеко-машинного интерфейса за счет использования принципа мета-схемного построения. В общем случае от разработчика прикладного сервиса не требуется совершать никаких дополнительных действий по созданию графических экранов. Интерфейс управления и конфигурирования генерируется для каждого сервиса автоматически на основе его предметной модели. Платформа транслирует модель сервиса в промежуточную схему данных, которая затем динамически интерпретируется веб-интерфейсом для построения элементов ввода и валидации параметров. Все сгенерированные формы автоматически собираются в единый веб-портал оператора, предоставляющий удаленный многопользовательский доступ к управлению комплексом из любой точки локальной сети.

Для обеспечения сквозной наблюдаемости в этот же портал интегрируются рассмотренные ранее панели Grafana, отображающие оперативные метрики сервисов в режиме реального времени. С помощью полученного веб-портала оператор осуществляет интерактивное конфигурирование параметров физических узлов и прикладных процессов. При сохранении настроек в интерфейсе инициируется обратный цикл синхронизации: платформа перехватывает введенные оператором данные, валидирует их на соответствие ограничениям предметной модели и автоматически фиксирует изменения обратно в центральный репозиторий. Это позволяет сохранить декларативный подход и гарантирует, что репозиторий всегда остается единственным и неделимым источником истины о состоянии системы. После фиксации ресурса в репозитории запускается стандартный процесс приведения оперативного состояния к новому административному, описанный ранее. Информационные потоки портала оператора представлены на рисунке 6.

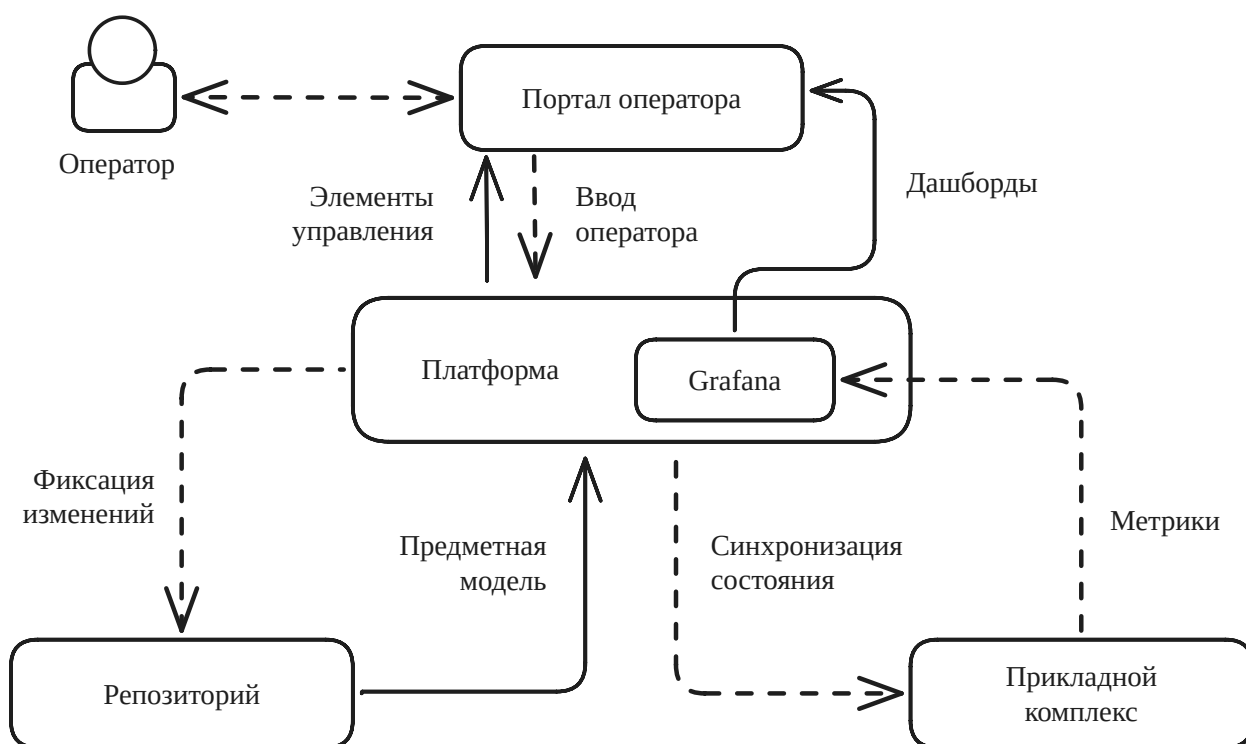


Рисунок 6 – Информационные потоки портала оператора

Для сценариев, когда автоматической генерации стандартных полей ввода недостаточно и требуется сложная, нестандартная разметка графического экрана (например, мнемосхема ускорительного кольца или интерактивное расположение датчиков), платформа расширяет предметную модель дополнительной абстракцией. Пользователь может декларативно описать структуру разметки (компоновку графических блоков) средствами мета-модели.

Используя эту абстракцию, разработчик может компоновать сложную панель управления из автоматически сгенерированных форм, потоковых графиков и специализированных виджетов. Сами интерактивные виджеты, за исключением типовых, разрабатываются индивидуально в виде изолированных веб-компонентов и регистрируются в реестре платформы для последующего переиспользования. Таким образом, оператор получает собственный функциональный интерфейс, который по-прежнему полностью управляется декларативно через текстовые файлы репозитория.

2.4. СЦЕНАРИИ ВЗАИМОДЕЙСТВИЯ

Рассмотрим особенности практической реализации прикладных сценариев взаимодействия, формализованных в подразделе 1.1.3, с учетом предложенных архитектурных решений:

- Устранение необходимости ручной разработки стандартных элементов интерфейса пользователя: экранные формы управления и конфигурирования генерируются динамически на основе декларативных схем моделей данных.
- Унифицированная регистрация новых функциональных блоков: для каждого сервиса формируется декларативный дескриптор развертывания, определяющий целевую среду выполнения и сопутствующие артефакты.
- Синхронизация ресурсов: все административные сценарии унифицируются и выполняются платформой.
- Полная автоматизация ввода сервисов в эксплуатацию: после добавления дескриптора развертывания в репозиторий и завершения автоматической синхронизации новый функционал становится доступным на портале оператора без проведения дополнительных ручных настроек.
- Централизованный мониторинг и управление: конфигурирование и контроль состояния всех запущенных сервисов осуществляются оператором через единый портал, предоставляющий прозрачный и унифицированный доступ ко всем функциональным модулям платформы.

3. РЕАЛИЗАЦИЯ

Данная глава посвящена техническому описанию реализации компонентов разработанной программной платформы и демонстрации её практической применимости в условиях научно-исследовательской лаборатории. В соответствии с концептуальными принципами, изложенными в предыдущей главе, платформа призвана инкапсулировать операционную сложность современных инструментов автоматизации, предоставляя пользователям лаконичный предметно-ориентированный интерфейс.

Описывать все разработанные программные модули, конфигурационные файлы и системные скрипты в отрыве от контекста эксплуатации не имеет практического смысла. В связи с этим описание инженерной реализации платформы в данной главе структурировано вокруг единого сквозного пользовательского сценария. Будут последовательно пройдены все этапы жизненного цикла прикладного программного обеспечения: начиная от организации унифицированной среды разработки и кодогенерации, заканчивая автоматизированным развертыванием разнородных служб в распределенном кластере и выводом потоковых данных на портал оператора.

В рамках рассматриваемого примера будут детально раскрыты внутреннее устройство разработанных подсистем платформы, принятые технические соглашения, компромиссы и пределы применимости текущей версии. Подобный подход позволяет наглядно продемонстрировать интеграцию всех компонентов платформы в единый, консистентный комплекс, способный эффективно функционировать в лабораторных условиях.

3.1. ВВЕДЕНИЕ К ПРИКЛАДНОМУ СЦЕНАРИЮ

Для демонстрации возможностей платформы по управлению гетерогенным программным обеспечением в рамках сквозного сценария был спроектирован и реализован комплекс из трех разнородных прикладных сервисов. Каждый из них решает изолированную физико-техническую задачу в контуре диагностики пучка

заряженных частиц и управления элементами магнитной системы коллайдера ВЭПП-2000.

Выбор сред выполнения, языков программирования и архитектуры потоков данных для этих сервисов обусловлен необходимостью моделирования трех принципиально разных инженерных парадигм, сосуществующих в реальной лабораторной инфраструктуре.

Ниже приведено описание разработанных демонстрационных сервисов:

- **Аппаратный сервис высокочастотной диагностики:**

- ▶ Назначение: Регистрация амплитуды напряжения с высокочастотного (ВЧ) резонатора ускорительного комплекса.
- ▶ Техническая реализация: Прошивка для микроконтроллера архитектуры AVR, функционирующая без операционной системы.
- ▶ Характер данных: Низколатентный управляющий контур. Сервис осуществляет скоростной аналогово-цифровой опрос датчиков, мгновенное побитовое кодирование телеметрии и высокочастотную отправку метрик в шину данных платформы.

- **Потоковый сервис оптической диагностики пучка:**

- ▶ Назначение: Непрерывная динамическая оценка эмиттанса и энергетического разброса пучка заряженных частиц на основе анализа его геометрических размеров, регистрируемых быстрыми CCD-камерами оптической диагностики. Формальная постановка задачи находится в Приложении А.
- ▶ Техническая реализация: Сервис на языке C++, упакованный в изолированный Docker-контейнер.
- ▶ Характер данных: Аналитический потоковый контур высокой пропускной способности. Сервис непрерывно потребляет кадры и сырые координатные массивы из шины данных, выполняет математическую аппроксимацию профиля пучка в реальном времени и транслирует вычисленные физические параметры обратно в шину для отображения на панелях мониторинга Grafana.

- **Пакетный сервис численного моделирования магнитных полей:**

- Назначение: Проведение тяжелых пространственных расчетов конфигурации магнитного поля многокатушечного корректирующего соленоида ускорителя при изменении токов в его обмотках. Формальная постановка задачи находится в Приложении Б.
- Техническая реализация: Сервис пакетной обработки данных на языке Python, разворачиваемый в виде Docker-контейнера.
- Характер данных: Дискретный контур. В отличие от предыдущих сервисов, данный модуль работает в прерывистом режиме: он запускается платформой только по запросу оператора, выполняет итерационный численный расчет и сохраняет финальные результаты в виде статических матриц и графических изображений в файловом хранилище портала.

Таким образом, представленный комплекс сервисов покрывает весь спектр возможных задач автоматизации лаборатории. В следующих разделах будет продемонстрировано, как разработанная платформа позволяет единообразно описывать, собирать и эксплуатировать столь разнородные программные модули через единый декларативный интерфейс репозитория.

3.2. ДЕКЛАРАТИВНОЕ ОПИСАНИЕ КОМПЛЕКСА

Определив физические задачи для трех целевых сервисов (диагностика пучка, управление резонатором и расчет соленоида), необходимо зафиксировать их параметры и ограничения до начала разработки прикладного кода. Задача этапа – перевести физические законы и аппаратные ограничения коллайдера ВЭПП-2000 на язык предметной модели.

В качестве базового инструмента для формализации предметной модели комплекса применяется декларативный язык конфигурации CUE [20]. Выбор данной технологии обусловлен её фундаментальной концепцией, построенной на математическом аппарате теории решеток и интуитивно понятной алгебре множеств. В отличие от традиционных форматов сериализации (таких как JSON или

YAML), CUE позволяет инженеру описывать физические инварианты и аппаратные ограничения системы не в виде императивных скриптов валидации, а через строгие логические операции объединения, пересечения и сужения типов. Такой подход делает конфигурацию самодокументируемой и гарантирует высокую надежность: любые противоречия в параметрах разнородных сервисов выявляются компилятором еще на этапе статического анализа, математически гарантируя, что система никогда не выйдет за рамки множества допустимых состояний.

Каждый из трех сервисов наследует базовые абстракции `dom.#Service` и `dom.#Config`, описанные в разделе 2.2, и дополняет их специфичными физическими инвариантами.

- Устройство для вычитывания напряжения (Сервис 1):

```
#Voltage: {  
  value: >=0  
}  
  
// Это устройство, которое не принимает конфигурацию,  
// а значит у него нет формального объекта "Config".  
// В данном случае его состояние - только исходный код.  
dom.#Service & {  
  driver: "firmware"  
}
```

- Поточковый сервис оценки (Сервис 2):

```
#AxisSetup: {  
  weight: >=0.0 & <=1.0 | *0.0  
}  
  
#AxesSetup: {  
  x: #AxisSetup  
  z: #AxisSetup  
}  
  
#CamSetup: {  
  axes: #AxesSetup  
}  
  
#Config: dom.#Config & {  
  // Окно потокового усреднения и расчета.
```

```

window: >0 | *3

// Параметры итерационного решателя.
solver: itr.#SolverConfig

cams: {
    // Объекта "ccd.cameras" перечисляет CCD камеры
    // используемые в лаборатории. Для каждой из
    // них сервис должен знать вес, чтобы провести
    // оптимизацию.
    for name in ccd.cameras {
        "\({name})"?: #CamSetup
    }
}

dom.#Service & {
    driver: "docker"
}

```

- Пакетный сервис расчета поля (Сервис 3) приведен ранее, в разделах 2.1.2 и 2.2.

Этот лаконичный код формирует «конституцию» экспериментального комплекса. Платформа математически гарантирует, что ни оператор через веб-интерфейс, ни смежный скрипт не смогут установить отрицательное окно измерений.

Чтобы превратить эти абстрактные математические ограничения в исполняемые прошивки AVR, контейнеры C++ и скрипты Python, необходимо предоставить пользователю унифицированную рабочую среду.

3.3. ОБЕСПЕЧЕНИЕ ВОСПРОИЗВОДИМОСТИ

Требуется получить полностью изолированное, воспроизводимое рабочее пространство разработчика на базе технологии Devcontainers и утилиты автоматизации Task [21]. Новый сотрудник должен запустить и собрать все три гетерогенных сервиса (AVR, C++, Python), установив на хост-машину только Docker и редактор кода.

3.3.1. ИЗОЛЯЦИЯ ИНСТРУМЕНТАРИЯ

Рабочее пространство полностью изолировано внутри контейнера стандарта Devcontainers на базе Ubuntu. Все разнородные тулчейны упакованы в виде модулей декларативного манифеста `.devcontainer/devcontainer.json`.

Ключевой фрагмент конфигурации среды:

```
"features": {  
  // Python-окружение  
  "ghcr.io/rocker-org/devcontainer-features/miniforge:2": {},  
  
  // AVR и C++ тулчейны  
  "ghcr.io/devcontainers-extra/features/apt-get-packages:1": {  
    "packages": "gcc-avr,avr-libc,cmake,gdb"  
  },  
  
  // Язык предметной модели  
  "ghcr.io/blabtm/features/cue:0.0.2": {}  
}
```

Данная настройка решает задачу гетерогенности:

- Кросс-компиляция AVR (Сервис 1): Инструменты кросс-компиляции позволяют собирать бинарный файл прошивки устройства.
- Сборка C++ (Сервис 2): Среда для компиляции потокового сервиса оптической диагностики пучка (см. [22]).
- Изоляция Python (Сервис 3): Фиксируется Python-окружение для расчета магнитного поля соленоида, защищая от конфликтов с хост-системой.
- Утилиты платформы: инструменты, необходимые для взаимодействия с платформой.

3.3.2. УНИФИКАЦИЯ ИНТЕРФЕЙСА АВТОМАТИЗАЦИИ

Чтобы избавить разработчика от ручного ввода длинных команд кросс-компиляции и сборки контейнеров, в платформу интегрирован инструмент автоматизации Task.

Глобальный интерфейс запуска в корне монорепозитория выглядит следующим образом:

```

includes:
  // Инструкции для сервиса 1
  hfr-v:    ./native/hfr_v/Taskfile.yml

  // Инструкции для сервиса 2
  em-es:    ./native/em_es/Taskfile.yml

  // Инструкции для сервиса 3
  sol-mf:   ./py/sol/mf/Taskfile.yml

tasks:
  build:
    desc: "Параллельная компиляция всех гетерогенных сервисов"
    deps:
      - task: hfr-v:build
      - task: em-es:build
      - task: sol-mf:build

```

Интерфейс полностью инкапсулирует сложность сборки. Разработчику достаточно выполнить в терминале контейнера одну интуитивную команду:

```
task build
```

Глобальный сценарий перехватывает вызов и параллельно запускает три разные подсистемы сборки.

Платформа использует этот механизм для управления инфраструктурой. Так, чтобы разработчику или администратору развернуть платформу целиком, требуется ввести одну команду:

```
task platform:deploy
```

Внутри скрыта конфигурация узлов с помощью Ansible. Описание узлов который платформа берет из предметной модели (см. раздел 2.2) и конвертирует в требуемый инвентарь. После преобразования предметной модели, описанной в разделе 2.2, инвентарь выглядит следующим образом:

```

cluster:
  hosts:
    master:
      ansible_host: 192.168.0.37
      ansible_port: 22

```


Теперь окружение полностью воспроизводимо и готово к разработке гетерогенных сервисов.

3.4. ТРАНСПОРТНАЯ ИНФРАСТРУКТУРА

Необходимо организовать динамический обмен телеметрией между микроконтроллером (Сервис 1) и потоковым анализатором (Сервис 2). Требуется определить топологию шины данных и единый стандарт упаковки сообщений. Платформа предоставляет двухконтурную транспортную инфраструктуру:

- Контур оперативного управления. Легковесный встраиваемый брокер MQTT на базе решения Mochi [23] для обмена данными с маломощным AVR-контроллером.
- Аналитический контур: Высокопроизводительная шина Redpanda для передачи видеок кадров и массивов координат между C++ контейнерами оптической диагностики, буферизации и временного хранения.

Подобная топология активно применяется в научных областях, включая системы мониторинга сейсмической активности, таких как QuakeFlow ([14]).

Если предметная модель описывает множество возможных состояний, то для описания данных в движении (телеметрии) платформа вводит понятие вычислительной модели. С помощью вычислительной модели описывается структура передаваемых сетевых пакетов, оптимизированных для бинарной сериализации.

Единым стандартом вычислительной модели в платформе выступает Protocol Buffers, который решает главную задачу гетерогенной среды: независимость от языка программирования.

Например, структуры пакетов для первых двух сервисов описываются в вычислительной модели следующим образом:

- Устройство для вычитывания напряжения (Сервис 1):

```
// Это будет на выходе контроллера.  
message Voltage {  
    int32 value = 1;
```

```
}
```

- Поточковый сервис оценки (Сервис 2):

```
// Это будет на входе сервиса.  
message AxisSize {  
    int64 time = 1;  
    double value = 2;  
}  
  
// Это будет на выходе сервиса.  
message Emittance {  
    int64 time = 1;  
    double current = 2;  
    double emittance = 3;  
}  
  
// Это будет на выходе сервиса.  
message EnergySpread {  
    int64 time = 1;  
    double current = 2;  
    double energy_spread = 3;  
}
```

Платформа требует, чтобы любая передача данных в транспортном контуре строго соответствовала контракту вычислительной модели. Это исключает ситуацию, при которой C++ сервис попытается прочитать строку, а устройство отправляет ему число.

Однако ручное написание вычислительной модели и ее синхронизация с предметной неизбежно приведет к рассинхронизации. Ограничения физического мира и форматы передачи данных должны генерироваться из единого источника истины.

3.5. КОНВЕЙЕР АВТОМАТИЗАЦИИ И КОДОГЕНЕРАЦИИ

Чтобы исключить человеческий фактор и предотвратить рассинхронизацию между физическими ограничениями предметной области и структурами в коде,

инженеру необходимо настроить автоматическую генерацию бинарных контрактов и исходного кода для всех трех гетерогенных сервисов.

Важнейшим архитектурным решением является то, что разработчик не пишет скрипты сквозной компиляции и не вызывает генераторы вручную. Вся техническая последовательность инкапсулирована внутри локальных сценариев.

В качестве примера рассмотрим фрагмент автоматизации для генерации C++ структур для потокового сервиса CCD-камер (Сервис 2):

```
tasks:
# Структуры для чтения конфигурации.
gen:config:
  cmds:
    - cue export ./config.cue -e '#Config' --out jsonschema \
      > /tmp/schema.json
    - sed -i -e 's/%23/#/g' /tmp/schema.json
    - |
      quicktype                    \
        --src /tmp/schema.json     \
        --src-lang schema         \
        --lang cpp                 \
        --out include/config.h

# Структуры транспорта.
gen:schema:
  cmds:
    - protoc -I{{.SCHEMA_DIR}} v2k/beam/AxisSize.proto
    - protoc -I{{.SCHEMA_DIR}} v2k/beam/Current.proto
    - protoc -I{{.SCHEMA_DIR}} v2k/beam/Emittance.proto
    - protoc -I{{.SCHEMA_DIR}} v2k/beam/EnergySpread.proto
```

Чтобы инициировать генерацию кода для всего комплекса целиком, достаточно выполнить в терминале одну унифицированную команду:

```
task gen
```

Платформа валидирует предметную модель и мгновенно раскладывает готовые артефакты по соответствующим директориям, гарантируя, что исполняемый код физически не способен разойтись с архитектурным замыслом.

После выполнения команды `task build`, описанной ранее, прошивка располагается в отведенном месте, а Docker-образы загружены в регистр и готовы к развертыванию. Информация из регистра приведена на рисунке 7.

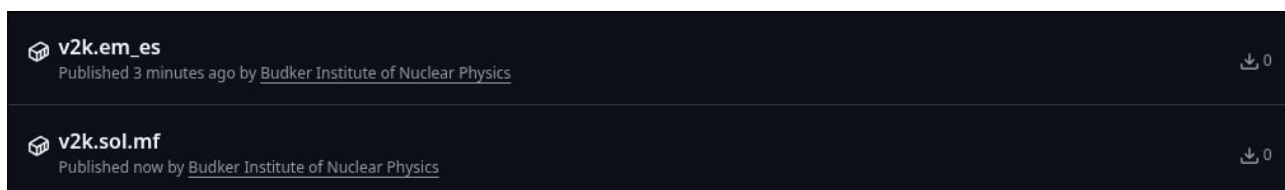


Рисунок 7 – Автоматически собранные и опубликованные сервисы

3.6. МЕНЕДЖЕР СЕРВИСОВ И СРЕДА ВЫПОЛНЕНИЯ

Стоит задача: запустить два сервиса (поточный C++ контейнер и пакетный контейнер) в вычислительном кластере лаборатории. При этом платформа должна гарантировать, что оперативное состояние этих процессов никогда не разойдется с административным замыслом, зафиксированным в предметной модели.

В условиях малых научных лабораторий развертывание тяжеловесных промышленных оркестраторов (таких как Kubernetes или HashiCorp Nomad) нецелесообразно из-за огромных накладных расходов на процессорное время и оперативную память. Для решения задачи унифицированного управления процессами был спроектирован легковесный системный модуль — менеджер сервисов.

Этот компонент выступает интеллектуальным контроллером платформы, реализуя парадигму GitOps [1] и изолируя конечного пользователя от специфики работы с Docker Swarm [24], SystemD или аппаратными интерфейсами.

3.6.1. КОНЦЕПЦИЯ

Фундаментальным архитектурным решением при проектировании менеджера сервисов стал полный отказ от использования встроенной базы данных. Менеджер спроектирован по принципу отсутствия состояния. Единственным источником истины о том, какие сервисы должны быть запущены, является центральный Git-репозиторий платформы.

В случае критического сбоя управляющего узла кластера или перезапуска самого менеджера, система не теряет данные: при старте менеджер вычитывает административное состояние из локальной копии репозитория, опрашивает нижележащие среды выполнения (Docker, SystemD) и мгновенно восстанавливает карту состояния комплекса. Это гарантирует, что сам менеджер сервисов не является точкой отказа.

Для управления процессами менеджер предоставляет унифицированный REST API и консольную утилиту, через которую подаются команды управления. Ниже приведено описание REST-интерфейса менеджера:

- GET /api/ls — получение списка зарегистрированных сервисов.
- GET /api/{name}/ps — получение оперативного состояния сервиса.
- GET /api/{name} — получение текущей конфигурации сервиса.
- PUT /api/{name} — установка новой конфигурации сервиса.
- PUT /api/{name}/fork — ветвление сервиса.
- GET /api/{name}/ui — генерация схемы для портала оператора.

3.6.1.1. ИНТЕРФЕЙС КОМАНДНОЙ СТРОКИ (CLI)

Для оперативного взаимодействия с менеджером сервисов из интерфейса командной строки разработана специализированная консольная утилита (CLI).

Для обеспечения атомарности выполнения управляющих операций и централизованного аудита действий, консольная утилита управления осуществляет вызовы исключительно к единому REST-интерфейсу менеджера сервисов. Подобная топология исключает прямое обращение клиентов к драйверам сред выполнения и гарантирует унификацию маршрутов обработки команд. Дополнительным преимуществом является возможность распространения предкомпилированной CLI-утилиты на все рабочие станции и виртуальные машины комплекса, обеспечивая удобный доступ для администраторов.

Разработанная утилита командной строки `sm` обладает интуитивным интерфейсом, воспроизводящим основные команды менеджера. Ниже представлены

примеры вывода справочной информации и списка зарегистрированных сервисов:

```
# sm
Service manager for the VEPP-2000 collider platform.
```

```
Usage:
  sm [command]
```

Available Commands:

```
  completion  Generate the autocompletion script for the specified
shell
  down        Take the service offline.
  help        Help about any command
  ls          List registered services.
  ps          Get the current operational status of the service.
  up          Take the service online.
```

Flags:

```
-c, --config string      configuration file
-h, --help               help for sm
    --remote.host string  remote host
    --remote.port int     remote port
```

Use "sm [command] --help" for more information about a command.

```
# sm ls
* em_es
* sol.mf
```

Как видно из приведенного выше фрагмента, как только сервисы попали в предметную модель, менеджер сразу распознал их. Для получения оперативного состояния, пользователи используют команду ps:

```
$ sm ps em_es
em_es:
  State: STOPPED

$ sm ps sol.mf
sol.mf:
  State: STOPPED
```

Для получения текущего административного состояния, используется команда get:

```

$ sm get sol.mf
active: true
coils:
  comp:
    currentAmp: 107
    material:
      relativePermeability: 1
    numberOfTurns: 4493
  nbsn:
    currentAmp: 107
    material:
      relativePermeability: 1
    numberOfTurns: 4367
  nbt1:
    currentAmp: 180
    material:
      relativePermeability: 1
    numberOfTurns: 8710
yoke:
  relativePermeability: 1000

```

3.6.2. УНИФИКАЦИЯ СРЕД ВЫПОЛНЕНИЯ

Чтобы управлять разнородными сервисами через один интерфейс, менеджер использует концепцию драйверов сред выполнения, позаимствованную у Nomad [25]. На уровне исходного кода языка Go это реализовано через строгий программный контракт:

```

type Driver interface {
    Ps(ctx context.Context, spec *Spec) (*Status, error)
    Up(ctx context.Context, spec *Spec, opts ...Option) error
    Down(ctx context.Context, spec *Spec) error
}

```

Когда инженер через консоль подает команду запуска сервиса, менеджер считывает дескриптор из предметной модели и динамически маршрутизирует вызов нужному драйверу:

- Для потокового (Сервис 2) и пакетного (Сервис 3) сервиса вызывается драйвер Docker Swarm [24], который транслирует параметры в вызовы Docker API и разворачивает изолированный контейнер.

- Для встраиваемых устройств вместо классического запуска процесса, драйвер MQTT отправляет бинарный пакет с административным состоянием напрямую в шину устройства, заставляя прошивку контроллера обновить свои параметры в реальном времени.

Чтобы изолировать ядро платформы и конечного пользователя от специфики жизненного цикла различных систем (например, кодов возврата оркестратора Docker Swarm или статусов системного демона SystemD), менеджер сервисов вводит унифицированную абстракцию оперативного состояния. Любой драйвер среды выполнения обязан транслировать внутреннее состояние подчиненного процесса (возвращаемое методом Ps) в одно из стандартизированных значений конечного автомата платформы:

- **STOPPED** — процесс штатно остановлен или еще не был инициализирован (соответствует административному замыслу `active: false`).
- **PENDING** — переходное состояние: сервис находится в процессе выделения ресурсов, загрузки контейнерного образа, сборки или доставки конфигурационного пакета по шине MQTT.
- **RUNNING** — сервис успешно запущен и непрерывно функционирует (типичное рабочее состояние для потокового аналитического контура и аппаратных драйверов устройств).
- **COMPLETED** — сервис штатно завершил выполнение конечной задачи (характерно для пакетных транзакционных задач, таких как расчет магнитного поля соленоида).
- **FAILED** — сервис завершил работу с ошибкой, был аварийно остановлен операционной системой или не смог запуститься из-за нехватки вычислительных ресурсов кластера.

Диаграмма состояний изображена на рисунке 8.

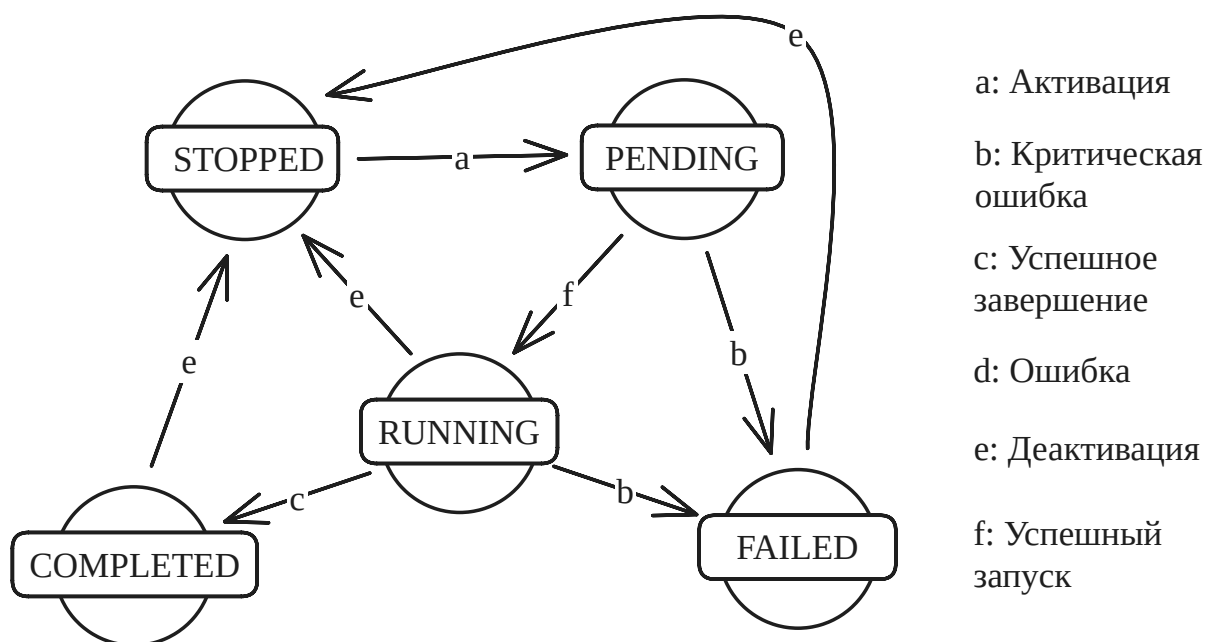


Рисунок 8 – Диаграмма состояний прикладного сервиса

Благодаря этой абстракции портал оператора, CLI-клиент и внутренние механизмы валидации оперируют единым словарем терминов, обеспечивая консистентный мониторинг и управление независимо от того, развернут ли сервис на микроконтроллере или внутри Docker-контейнера.

Для запуска Сервиса 2, нужно установить в его административное состояние параметр `active` в значение `true`. Менеджер сервисов выполняет глубокое слияние переданного состояния с тем, что уже есть:

```
$ echo '{"active": true}' | sm set em_es
OK
```

```
$ sm ps em_es
em_es:
  State: PENDING
```

```
$ sm ps em_es
em_es:
  State: RUNNING
```

Сервисы могут одновременно существовать в нескольких экземплярах. Функционал создания новой сущности реализован в методе `fork`. Рассмотрим на примере сервиса расчета магнитного поля:

1. Сервис с именем `sol.mf` регистрируется на платформе по универсальной процедуре.
2. При получении запроса `fork` на ветвление, менеджер сервисов генерирует уникальный идентификатор выполнения `iid` (KSUID), содержащий текущую временную метку.
3. Создается дополнительная директория `{iid}` рядом с базовой конфигурацией.
4. Переданная в запросе конфигурация сливается с базовой и размещается в новой директории `{iid}`.
5. Создается Swarm-сервис с идентификатором `sol_mf_{iid}`. Передается переменная окружения `IID={iid}`. По ней сервис находит свою конфигурацию и создает соответствующую директорию с результатами.
6. Дальнейшие запросы к менеджеру сервисов сопровождаются дополнительным параметром `iid`.

```
$ sm fork sol.mf
OK: 20260616_104627
```

```
$ sm ps sol.mf 20260616_104627
sol.mf:
  State: COMPLETED
```

```
$ ls -l config/sol/mf
drwxr-xr-x. ... 20260603_090753
drwxr-xr-x. ... 20260603_092545
dr-xr-xr-x. ... 20260616_104627 // Последний запуск
-rw-r--r--. ... config.json      // Базовая конфигурация.
```

3.6.3. ДВУНАПРАВЛЕННАЯ СИНХРОНИЗАЦИЯ

Платформа запрещает ручное изменение конфигурационных файлов (это может делать только авторизованный персонал для разрешения конфликтов). Менеджер сервисов замыкает петлю автоматической синхронизации.

Когда пользователь меняет состояние сервиса, менеджер выполняет следующую последовательность:

- Валидирует конфигурацию на соответствие ограничениям предметной модели.
- Модифицирует локальный конфигурационный артефакт.
- Автоматически формирует Git Commit с описанием изменений.
- Отправляет изменения в центральный репозиторий.
- Только после успешной фиксации в истории версий Менеджер отдает команду драйверу среды выполнения на физический запуск процесса.

После последнего запуска сервиса `sol.mf`, состояние репозитория изменилось:

```
$ git log
```

```
commit 97fec6e020e15f362f980e0d206cfc44fbadce8f (HEAD -> trunk)
Author: Service Manager <support@blab.org>
Date: Tue Jun 16 10:46:27 2026 +0700
```

```
sm: configuration update
```

```
...
```

3.6.4. ВАЛИДАЦИЯ И ПАТТЕРН «GATEKEEPER»

Ключевая инновация менеджера сервисов заключается в программной интеграции компилятора языка CUE. Менеджер не просто перекладывает файлы, он выполняет роль валидатора состояния лабораторного комплекса.

Рассмотрим сценарий защиты от операторской ошибки. Инженер через CLI пытается применить конфигурацию для аналитического сервиса (Сервис 2), указав параметр `window` равным `-1`. Менеджер сервисов перехватывает этот артефакт и валидирует его относительно предметной модели, в которой (как мы определили ранее) задан жесткий физический лимит `window: >0`.

Проверка завершается с ошибкой. В этот момент срабатывает алгоритм защиты:

- Новое состояние сохраняется, но не фиксируется.

- Система переходит в режим блокировки: менеджер сервисов замораживает целевой сервис в последнем известном корректном состоянии и создает файл блокировки в репозитории (.lock).
- Любая дальнейшая автоматическая синхронизация для этого сервиса блокируется до тех пор, пока администратор платформы вручную не расследует инцидент и не исправит некорректные параметры, удалив файл блокировки из репозитория.

Диаграмма последовательности изображена на рисунке 9.

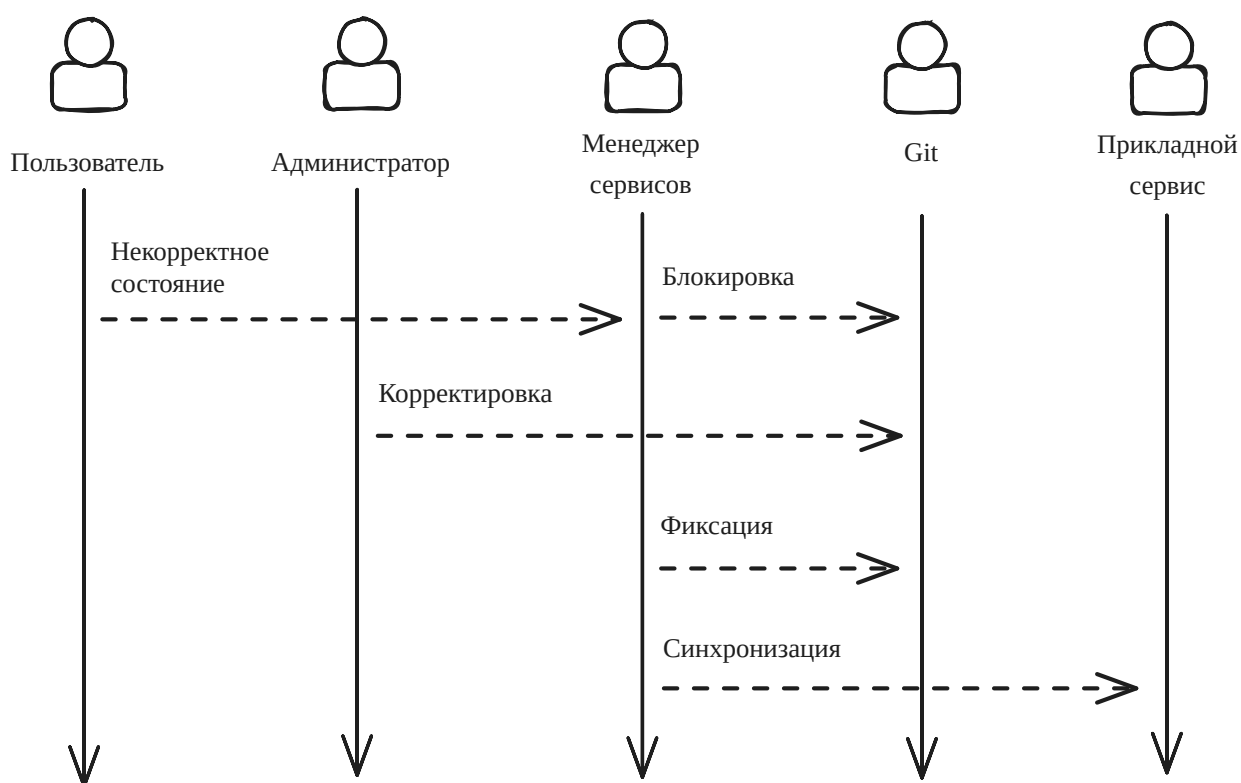


Рисунок 9 – Диаграмма последовательности блокировки

Это исключает риск повреждения дорогостоящего экспериментального оборудования из-за некорректных команд или программных сбоев внешних скриптов.

Очевидным финальным шагом является необходимость предоставить оператору коллайдера визуальный интерфейс для управления этими запущенными процессами и наблюдения за потоками данных.

3.7. ПОРТАЛ ОПЕРАТОРА

Необходимо предоставить оператору инструмент для управления комплексом. При этом оператор не должен иметь прямого доступа к конфигурационным файлам в репозитории или консоли сервера.

Традиционный подход предполагал бы ручную разработку графических панелей управления (HMI) для каждого нового сервиса с использованием фреймворков вроде Qt или других библиотек. Однако в условиях ограниченных кадровых ресурсов подход платформы исключает ручную верстку типовых экранов.

Разработанный веб-портал оператора реализует принцип мета-схемной генерации на базе библиотеки React и генератора JsonForms [26]. Портал не содержит жестко запрограммированных полей ввода. Вместо этого при обращении к сервису портал загружает его промежуточную схему, сгенерированную из предметной модели менеджером, и динамически отрисовывает элементы управления. Генерация схемы происходит динамически: при изменении предметной модели, оператору потребуется только обновить страницу.

Для непрерывно работающего сервиса оптической диагностики пучка портал генерирует комбинированный интерфейс. На основе ограничений предметной модели автоматически строятся поля ввода, а для наблюдения за результатами в интерфейс прозрачно интегрируется панель Grafana (см. рисунок А.1). Сгенерированный интерфейс изображен на рисунке 10.

← → ↻ ⚠ Not secure 192.168.0.37:5173/em_es ☆ 📁 👤 ⋮

SET STOPPED

☐ Active Window 4

Solver

Absolute Cost 1e-7

Max Evaluations 1000

Max Iterations 1000

Relative Cost 1e-10

CCD

1 M 1 L

Axes X

Weight 1

1 M 1 R

Axes X

Weight

Рисунок 10 – Панель управления для потокового сервиса

Интегрированная панель Grafana напрямую подписана на аналитический контур Redpanda, отображая бинарные пакеты Protobuf (энергетический разброс и эмиттанс) в реальном времени, используя внедренные ранее схемы данных.

Взаимодействие с сервисом расчета магнитного поля соленоида строится по транзакционной модели. Сгенерированная форма предлагает оператору задать параметры для расчетов. Платформа блокирует ввод значений, превышающих

физические ограничения, описанные в предметной модели. Сгенерированный интерфейс изображен на рисунке 11.

← → ↻ ⚠ Not secure 192.168.0.37:5173/sol.mf ☆ 📁 👤 ⋮

SET **STOPPED**

☒ Active

Coils

Comp

Current Amp — 107

Material

Density

Relative Permeability — 1

Number Of Turns — 4493

Nbsn

Current Amp — 107

Material

Рисунок 11 – Панель управления для пакетного сервиса

3.7.1. ЗАМЫКАНИЕ ЦИКЛА GITOPS

Важнейшей функцией веб-портала является обеспечение целостности GitOps-подхода. Когда оператор меняет любой параметр в сгенерированной форме и сохраняет его, портал выполняет следующую последовательность:

- Отправляет запрос менеджеру сервисов.
- Менеджер сервисов автоматически формирует коммит с измененными параметрами и отправляет его в центральный Git-репозиторий.
- Запускается процесс приведения оперативного состояния к новому административному.

Диаграмма последовательности изображена на рисунке 12.

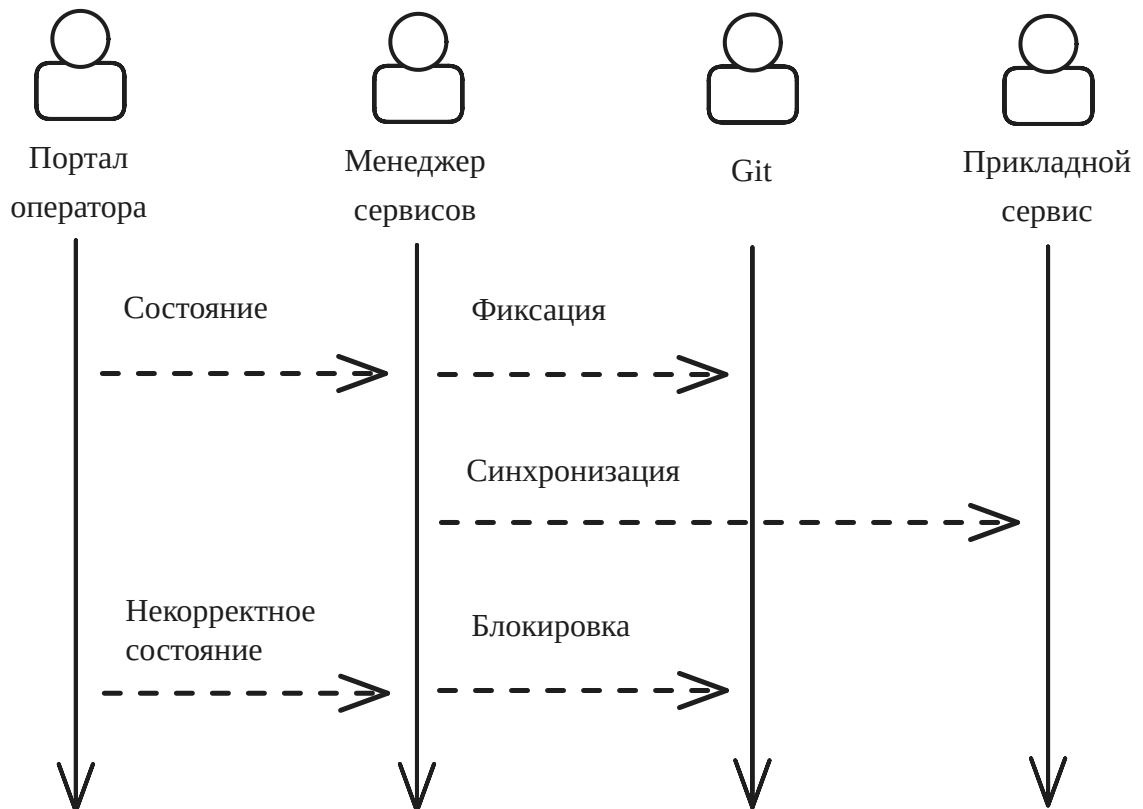


Рисунок 12 – Диаграмма последовательности фиксации состояния

Таким образом, оператор работает в удобном и безопасном веб-интерфейсе, в то время как репозиторий остается единственным, математически строгим источником истины о состоянии коллайдера.

3.8. ОГРАНИЧЕНИЯ РЕАЛИЗАЦИИ

Проектирование любой высоконагруженной распределенной программной системы в условиях реального времени сопряжено с поиском компромиссов между производительностью, надежностью, простотой сопровождения и стоимостью развертывания. Разработанный прототип программной платформы уско-

рительного комплекса обладает рядом технических ограничений и допущений, обусловленных спецификой физических установок и ограниченными аппаратными ресурсами:

- **Зависимость от централизованного NFS-хранилища.** Использование общей сетевой файловой системы NFS для монтирования глобальных конфигурационных файлов и реестра Protobuf-схем упрощает распространение артефактов, но создает единую точку отказа. В случае сбоя или сетевой недоступности управляющего узла, функционирующего как NFS-сервер, прикладные сервисы трансляции данных потеряют доступ к схемам, что приведет к остановке конвейера схематизации.
- **Ограничения автоматической генерации пользовательского интерфейса.** Применение библиотеки JsonForms на основе декларативных схем Cue позволяет мгновенно генерировать типовые формы управления и ввода параметров. Тем не менее, данная методология неприменима к сложным специализированным компонентам отображения (таким как интерактивные карты ускорительного кольца, графики фазовых портретов или трехмерные модели распределения полей), которые требуют ручной разработки на React и интеграции через реестр UI-компонентов.
- **Ручное выполнение команд автоматизации.** Несмотря на то, что сложные команды по синхронизации состояния инкапсулированы за упрощенным интерфейсом Task, их все еще приходится запускать вручную или средствами CI/CD.

Выделенные ограничения не являются критическими барьерами для эксплуатации платформы, однако очерчивают границы применимости текущей версии архитектуры и формируют список задач для будущих этапов модернизации комплекса.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы была спроектирована, программно реализована и проверена на практике программная платформа, предназначенная для разработки, развертывания и эксплуатации прикладных сервисов в научных лабораториях с ограниченными вычислительными ресурсами и малочисленным штатом. Практическая проверка разработанных программных решений осуществлена на базе экспериментальной инфраструктуры лаборатории №11 Института ядерной физики им. Г.И. Будкера СО РАН в контуре накопителя ВЭПП-2000.

В ходе проведения исследования и выполнения практической разработки были получены следующие результаты:

- Сформулированы требования к программной платформе, функционирующей в условиях ограниченности вычислительных ресурсов и специфики малочисленного штата специалистов.
- Разработана концепция декларативного управления целевым состоянием прикладного программного комплекса и разработан прототип программной платформы на основе методологии GitOps и предметно-ориентированного моделирования.
- Проведена практическая проверка работоспособности разработанной платформы на примере прикладных сервисов обработки данных в инфраструктуре лаборатории.

Разработанный прототип платформы представляет собой практический результат, направленный на повышение надежности, наблюдаемости и воспроизводимости прикладного программного комплекса физической установки. Созданный прототип платформы может служить готовым решением для комплексной модернизации ИТ-инфраструктуры в других исследовательских коллективах и лабораториях, сталкивающихся со схожими прикладными задачами и ресурсными ограничениями.

Перспективы дальнейшего развития работы лежат в плоскости расширения возможностей автоматической кодогенерации по декларативным спецификациям и масштабирования на другие задачи, возникающие в ходе эксплуатации ускорительного комплекса ВЭПП-2000.

СПИСОК ЛИТЕРАТУРЫ

1. Limoncelli T. A. GitOps: a path to more self-service IT // Commun. ACM. New York, NY, USA: Association for Computing Machinery, 2018. Т. 61, № 9. Сс. 38–42. URL: <https://doi.org/10.1145/3233241> (дата обращения: 14.04.2026).
2. Senchenko A., Berkaev D., Kasaev A. VEPP-2000 Collider Control System // Proc. PCaPAC'12. JACoW Publishing, Geneva, Switzerland. Сс. 263–267. URL: <https://jacow.org/pcapac2012/papers/FRCB04.pdf> (дата обращения: 14.04.2026).
3. Development Containers Specification : сайт / Development Containers Community. URL: <https://containers.dev/implementors/спес> (дата обращения: 14.04.2026).
4. Valstar S., Griswold W. G., Porter L. Using DevContainers to Standardize Student Development Environments: An Experience Report // Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education. Trondheim, Norway: Association for Computing Machinery, 2020. Сс. 377–383. URL: <https://doi.org/10.1145/3341525.3387424> (дата обращения: 14.04.2026).
5. Strukčinskas M. A., Badem S., Jokubauskas A. Devpod: Improving Developer Productivity at Uber with Remote Development // Uber Technologies Inc.: электрон. журн. URL: <https://www.uber.com/it/en/blog/devpod-improving-developer-productivity-at-uber>. Дата публикации: 13.12.2022.
6. What is Tailscale : сайт / Tailscale Inc. URL: <https://tailscale.com/docs/concepts/what-is-tailscale> (дата обращения: 14.04.2026).
7. Jaspan C., Jorde M., Knight A. Advantages and disadvantages of a monolithic repository: a case study at Google // Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice. Gothenburg, Sweden: Association for Computing Machinery, 2018. Сс. 225–234. URL: <https://doi.org/10.1145/3183519.3183550> (дата обращения: 14.04.2026).
8. Potvin R., Levenberg J. Why Google Stores Billions of Lines of Code in a Single Repository // Communications of the ACM. 2016. Т. 59. Сс. 78–87. URL: <http://dl.acm.org/citation.cfm?id=2854146> (дата обращения: 14.04.2026).

9. Humble J., Farley D. Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation. Addison-Wesley Professional, 2010.
10. Kubernetes: программа / Google LLC. URL: GitHub, <https://github.com/kubernetes/kubernetes> (дата обращения: 14.04.2026).
11. Terraform: программа / HashiCorp, Inc. URL: GitHub, <https://github.com/hashicorp/terraform> (дата обращения: 14.04.2026).
12. Dean J., Ghemawat S. MapReduce: simplified data processing on large clusters // Communications of the ACM. 2008. Т. 51, № 1. Сс. 107–113.
13. Akidau T., Bradshaw R., Chambers C. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing // Proceedings of the VLDB Endowment. 2015. Т. 8, № 12. Сс. 1792–1803.
14. Zhu W., Hou A. B., Yang R. QuakeFlow: a scalable machine-learning-based earthquake monitoring workflow with cloud computing // Geophysical Journal International. 2023. Т. 232, № 1. Сс. 684–693. URL: <https://doi.org/10.1093/gji/ggac355> (дата обращения: 14.04.2026).
15. Burns B., Grant B., Oppenheimer D. Borg, Omega, and Kubernetes // ACM Queue. 2016. Т. 14. Сс. 70–93. URL: <http://queue.acm.org/detail.cfm?id=2898444> (дата обращения: 14.04.2026).
16. Thota M. R. Intelligent Infrastructure as Code: GitOps, Resilience Engineering, and Automation Patterns for Scalable Big Data Platforms // International Journal of Scientific Research in Science and Technology. 2021. Т. 8. Сс. 741–752. URL: https://www.researchgate.net/publication/400918999_Intelligent_Infrastructure_as_Code_GitOps_Resilience_Engineering_and_Automation_Patterns_for_Scalable_Big_Data_Platforms (дата обращения: 14.04.2026).
17. Grafana: программа / Grafana Labs. URL: GitHub, <https://github.com/grafana/grafana> (дата обращения: 14.04.2026).
18. Jones T. How CERN uses Grafana and Mimir to monitor the world’s largest computer grid // Grafana Labs: электрон. журн. URL: <https://grafana.com/>

blog/how-cern-uses-grafana-and-mimir-to-monitor-the-worlds-largest-computer-grid. Дата публикации: 03.01.2025.

19. Observability as Code : сайт / Grafana Labs. URL: <https://grafana.com/docs/grafana/latest/as-code/observability-as-code> (дата обращения: 14.04.2026).
20. CUE: программа / The CUE Authors. URL: GitHub, <https://github.com/cue-lang/cue> (дата обращения: 14.04.2026).
21. Task: программа / The Task Authors. URL: GitHub, <https://github.com/go-task/task> (дата обращения: 14.04.2026).
22. CMake: программа / Kitware, Inc. URL: GitHub, <https://cmake.org/> (дата обращения: 14.04.2026).
23. Mochi MQTT Server: программа / The Mochi MQTT Authors. URL: GitHub, <https://github.com/mochi-mqtt/server> (дата обращения: 14.04.2026).
24. Docker: программа / Docker, Inc. URL: GitHub, <https://github.com/docker> (дата обращения: 14.04.2026).
25. Nomad: программа / HashiCorp, Inc. URL: GitHub, <https://github.com/hashicorp/nomad> (дата обращения: 14.04.2026).
26. JSON Forms: программа / EclipseSource. URL: GitHub, <https://github.com/eclipsesource/jsonforms> (дата обращения: 14.04.2026).
27. Rogovsky Y. A., Perevedentsev E., Shwartz D. Current Dependence of Bunch Dimensions in VEPP-2000 Collider // Proc. 26th Russian Particle Accelerator Conference (RUPAC'18), Protvino, Russia, 01-05 October 2018. Geneva, Switzerland: JACoW Publishing, 2018. № 26. Сс. 125–127. URL: <http://jacow.org/rupac2018/papers/thcemh04.pdf> (дата обращения: 14.04.2026).

ПРИЛОЖЕНИЕ А. ПОТОКОВЫЙ СЕРВИС

ФОРМУЛИРОВКА

Модуль предназначен для оценки эмиттанса и энергетического разброса пучка заряженных частиц. По своей сути модуль выполняет численную минимизацию заданной физической модели с использованием полученных измерений размера пучка в качестве наблюдений. Алгоритм разработан как для обработки данных в реальном времени, так и для воспроизведения эксперимента.

ФИЗИЧЕСКОЕ ОБОСНОВАНИЕ

Моделируется зависимость эмиттанса и энергетического разброса пучка заряженных частиц от его тока. Уравнения А.1 и А.2 описывают зависимость размеров пучка от параметров (см. [27]):

$$\sigma_x^2 = \beta_x \varepsilon + (D_x \sigma_e)^2, \quad (\text{А.1})$$

$$\sigma_z^2 = \beta_z \varepsilon + (D_z \sigma_e)^2, \quad (\text{А.2})$$

где $\sigma_{\{x,z\}}$ – размер пучка, $\beta_{\{x,z\}}$ – бета-функция, $D_{\{x,z\}}$ – дисперсия орбиты пучка, σ_e – энергетический разброс, ε – эмиттанс.

Допущения:

- Размер пучка в накопительном кольце определяется поперечным распределением частиц, которое является гауссовским при низких токах (микроамперы). Таким образом, под размером пучка подразумевается ширина гауссовского распределения.
- Измерения размера пучка не синхронизированы по времени и требуют дополнительной предварительной обработки.
- Измерения размера пучка имеют определённую техническую точность, определяемую:
 - фокусировкой изображения в плоскости CCD-сенсора;
 - размером зерна CCD-сенсора;

- ▶ алгоритмом аппроксимации изображения пучка двумерной гауссовой функцией;
- ▶ переэкспонированием CCD-сенсора;
- ▶ наличием бликов.

С учётом последнего пункта, для практических целей приведённые выше выражения могут быть переписаны в виде уравнений A.3 и A.4:

$$\sigma_x^2 - \beta_x \varepsilon + (D_x \sigma_e)^2 \geq 0, \quad (\text{A.3})$$

$$\sigma_z^2 - \beta_z \varepsilon + (D_z \sigma_e)^2 \geq 0. \quad (\text{A.4})$$

ПРИНЦИП РЕШЕНИЯ

Сохраняя знак равенства, проводится численная минимизация неявно заданной вектор-функции по интересующим параметрам (эмиттанс и энергетический разброс). Известные компоненты системы усредняются за некоторый настраиваемый интервал, после чего формируются в матрицу параметров. В качестве алгоритма минимизации использовался метод Гаусса-Ньютона.

РЕЗУЛЬТАТ

Результатом работы сервиса являются зависимости энергетического разброса и эмиттанса от тока. Пример визуализации результатов в Grafana приведена на рисунке A.1 (зависимость эмиттанса и энергетического разброса от тока соответственно):



Рисунок A.1 – Пример визуализации результатов в Grafana

Реализация алгоритма численной минимизации была протестирована на наборе данных, полученном в научной работе [27], использованной в качестве основы данного исследования. Корректность организации потоковых вычислений проверена на синтетических данных с различным показателем шума.

ПРИЛОЖЕНИЕ Б. ПАКЕТНЫЙ СЕРВИС

ФОРМУЛИРОВКА

Исследуется распределение величины продольного поля на оси многокатушечного соленоида.

Внутри железного ярма установлено 5 concentric токоведущих катушек. Вся конструкция имеет аксиальную симметрию. Необходимо найти распределение продольной компоненты магнитного поля вдоль оси системы и интеграл этой компоненты.

Схематично, исследуемая область изображена на рисунке Б.1:

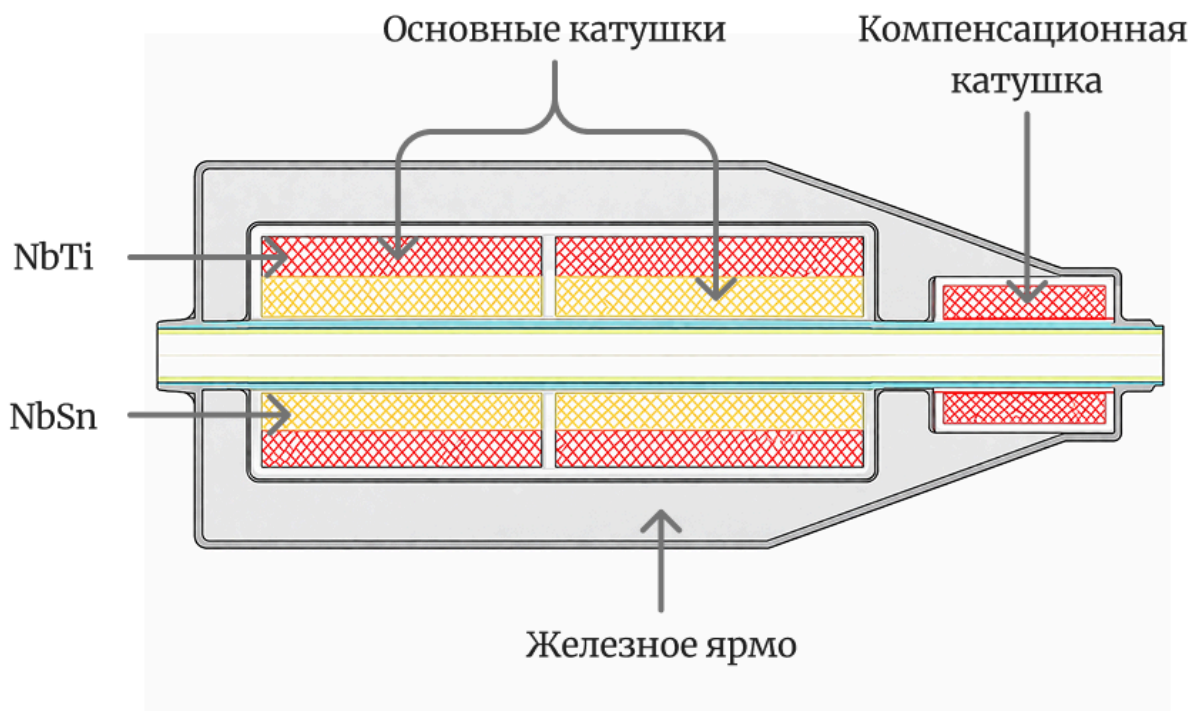


Рисунок Б.1 – Схематичное изображение многокатушечного соленоида

Для всей исследуемой области известны значения магнитной проницаемости материалов. Для катушек заданы количество витков и ток.

ПРИНЦИП РЕШЕНИЯ

Для решения поставленной магнитостатической задачи используется метод конечных элементов (МКЭ). Индукция магнитного поля представляется в

виде ротора некоторой вектор-функции, называемой вектор-потенциалом. Полученные математические соотношения приводятся к двумерному уравнению в цилиндрических координатах, что возможно благодаря аксиальной симметрии расчетной области.

Для построения конечноэлементной сетки используется открытый пакет GMSH. Полученная сетка с выделенными физическими группами изображена на рисунке Б.2:

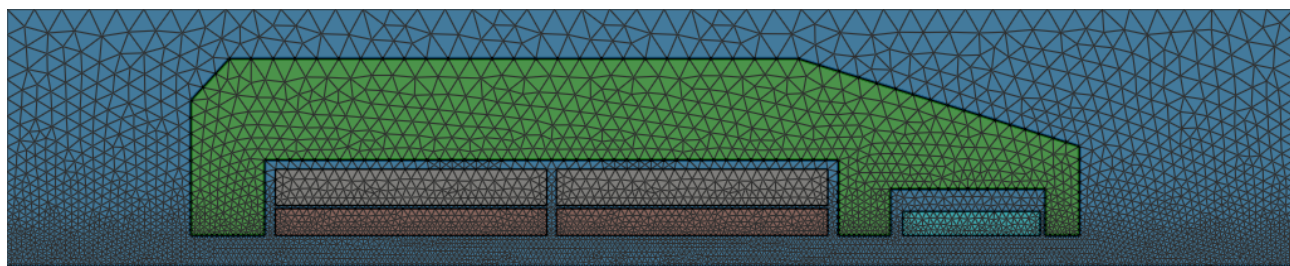


Рисунок Б.2 – Конечноэлементная сетка

В качестве реализации МКЭ используется открытый конечноэлементный пакет FEniCS. Для аппроксимации функции вектор-потенциала используется непрерывное функциональное пространство Галеркина первого порядка на треугольных элементах. Для аппроксимации поля используется разрывное пространство Галеркина нулевого порядка.

РЕЗУЛЬТАТ

На рисунке Б.3 приведено распределение продольной компоненты индукции магнитного поля:

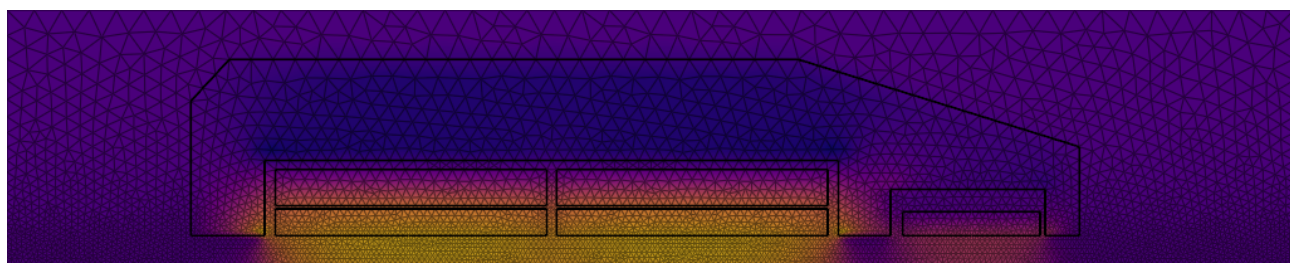


Рисунок Б.3 – Продольная компонента индукции магнитного поля

В результате измерения распределения магнитного поля, получен интеграл продольной компоненты на оси соленоида ≈ 7.643 Тл. Решение было проверено

на корректность глобальных физических интегралов, а именно расчетом значения магнитной энергии полевым методом и методом источника.